

It is worth mentioning (or admitting) that using repartition here is a bit of an anti-pattern. Explicit repartitioning with large datasets can have performance implications, especially if it causes a shuffle between nodes on the cluster. In most cases of calling a REST API, the data size of any micro-batch is not massive. So, in practical terms, this technique is unlikely to cause a problem. And, it has a big positive effect on throughput to the API.

4. Execute a dataframe transformation that calls a nested function dedicated to making a single call to the REST API. The input to this function will be one row of pre-batched data. In the sample, the payload column has the data to include on a single call. Call a dataframe action method to invoke execution of the transformation.

```
1 submitted_df = new_df.withColumn("RestAPIResponseCode",\
2                               callRestApiOnce(new_df["payload"])).\
3                               collect()
```

5. Inside the nested function which will make one API call, use your libraries of choice to issue an HTTP POST against the REST API. This is commonly done with the **Requests** library but any library suitable for making the call can be considered. See the `callRestApiOnce` method in Exhibit 1 for an example.
6. Handle potential errors from the REST API call by using a `try..except` block or checking the HTTP response code. If the call is unsuccessful, the overall job can be failed by throwing an exception (for job retry or troubleshooting) or individual records can be diverted to a dead letter queue for remediation or later retry.

```
1 if not (response.status_code==200 or response.status_code==201) :
2     raise Exception("Response status : {} .Response message : {}".\
3                     format(str(response.status_code),response.text))
```

The six elements above should prepare your code for sending streaming data to a REST API, with the ability to scale for throughput and to handle error conditions cleanly. The sample code in Exhibit 1 is an example implementation. Each point stated above is reflected in the full example.

```

1  from pyspark.sql.functions import *
2  from pyspark.sql.window import Window
3  import math
4  import requests
5  from requests.adapters import HTTPAdapter

6  def preBatchRecordsForRestCall(microBatchDf, batchSize):
7      batch_count = math.ceil(microBatchDf.count() / batchSize)
8      microBatchDf = microBatchDf.withColumn("content", to_json(struct(col("*"))))
9      microBatchDf = microBatchDf.withColumn("row_number",\
10                                         row_number().over(Window().
11 orderBy(lit('A'))))
12      microBatchDf = microBatchDf.withColumn("batch_id", col("row_number") % batch_
13 count)
14      return microBatchDf.groupBy("batch_id").\
15                          agg(concat_ws(",", collect_
16 list("content"))).\
17                          alias("payload"))

17 def callRestAPIBatch(df, batchId):
18     restapi_uri = "<REST API URL>"

19     @udf("string")
20     def callRestApiOnce(x):
21         session = requests.Session()
22         adapter = HTTPAdapter(max_retries=3)
23         session.mount('http://', adapter)
24         session.mount('https://', adapter)

24         #this code sample calls an unauthenticated REST endpoint; add headers
25         necessary for auth
26         headers = {'Authorization':'abcd'}
27         response = session.post(restapi_uri, headers=headers, data=x, verify=False)
28         if not (response.status_code==200 or response.status_code==201) :
29             raise Exception("Response status : {} .Response message : {}".\
30                             format(str(response.status_code),response.text))

31         return str(response.status_code)

```

```

32     ### Call helper method to transform df to pre-batched df with one row per REST
33     API call
34     ### The POST body size and formatting is dictated by the target API; this is an
35     example
36     pre_batched_df = preBatchRecordsForRestCall(df, 10)

37     ### Repartition pre-batched df for target parallelism of API calls
38     new_df = pre_batched_df.repartition(8)

39     ### Invoke helper method to call REST API once per row in the pre-batched df
40     submitted_df = new_df.withColumn("RestAPIResponseCode",\
41                                     callRestApiOnce(new_df["payload"])).collect()

42     dfSource = (spark.readStream
43                 .format("delta")
44                 .table("samples.nyctaxi.trips"))

45     streamHandle = (dfSource.writeStream
46                     .foreachBatch(callRestAPIBatch)
47                     .trigger(availableNow=True)
48                     .start())

```

Exhibit 1

DESIGN AND OPERATIONAL CONSIDERATIONS

Exactly Once vs At Least Once Guarantees

As a general rule in Structured Streaming, using `foreachBatch` only provides at-least-once delivery guarantees. This is in contrast to the exactly-once delivery guarantee provided when writing to sinks like a **Delta table** or file sinks. Consider, for example, a case where 1,000 records arrive on a micro-batch and your code in `foreachBatch` begins calling the REST API with the batch. In a hypothetical failure scenario, let's say that 900 calls succeed before an error occurs and fails the job. When the stream restarts, processing will resume by re-processing the failed batch. Without additional logic in your code, the 900 already-processed calls will be repeated. It is important that you determine in your design whether this is acceptable, or whether you need to take additional steps to protect against duplicate processing.

The general rule when using `foreachBatch` is that your target sink (REST API in this case) should be idempotent or that you must do additional tracking to account for multiple calls with the same data.

Estimating Cluster Core Count for a Target Throughput

Given these techniques to call a REST API with streaming data, it will quickly become necessary to estimate how many parallel executors/tasks are necessary to achieve your required throughput. And you will need to select a cluster size. The following table shows an example calculation for estimating the number of worker cores to provision in the cluster that will run the stream.

A	Target Throughput (records/sec)	3200	←Required rate of records processed by stream and submitted to REST API
B	# records batched per REST API call	10	←Average number of records included on POST body for each API call
C	Total required calls per second	320	A / B, rounded up
D	API Latency (ms to make one call)	50	← Plug in estimated or observed latency
E	Max calls per task per second	20	← 1000ms / D, rounded up
F	Min required parallelism	16	C / E, rounded up
G	Shared use factor	2	Rule-of-thumb multiplier to account for other work being done on cluster
H	Estimated core count	32	F * G, rounded up.

Line H in the table shows the estimated number of worker cores necessary to sustain the target throughput. In the example shown here, you could provision a cluster with two 16-core workers or 4 8-core workers, for example. For this type of workload, fewer nodes with more cores per node is preferred.

Line H is also the number that would be put in the `repartition` call in `foreachBatch`, as described in item 3 above.

Line G is a rule of thumb to account for other activity on the cluster. Even if your stream is the only job on the cluster, it will not be calling the API 100% of the time. Some time will be spent reading data from the source stream, for example. The value shown here is a good starting point for this factor – you may be able to fine tune it based on observations of your workload.

Obviously, this calculation only provides an estimated starting point for tuning the size of your cluster. We recommend you start from here and adjust up or down to balance cost and throughput.

OTHER FACTORS TO CONSIDER

There are other factors you may need to plan for in your deployment. These are outside the scope of this post, but you will need to consider them as part of implementation. Among these are:

- 1. Authentication requirements of the target API: It is likely that the REST API will require authentication. This is typically done by adding required headers in your code before making the HTTP POST.
- 2. Potential rate limiting: The target REST API may implement rate limiting which will place a cap on the number of calls you can make to it per second or minute. You will need to ensure you can meet throughout targets within this limit. You'll also want to be ready to handle throttling errors that may occur if the limit is exceeded.
- 3. Network path required from worker subnet to target API: Obviously, the worker nodes in the host Spark cluster will need to make HTTP calls to the REST API endpoint. You'll need to use the available cloud networking options to configure your environment appropriately.
- 4. If you control the implementation of the target REST API (e.g., an internal custom service), be sure the design of that service is ready for the load and throughput generated by the streaming workload.

MEASURED THROUGHPUT TO A MOCKED API WITH DIFFERENT NUMBERS OF PARALLEL TASKS

To provide representative data of scaling REST API calls as described here, we ran tests using code very similar to Example 1 against a mocked up REST API that persisted data in a log.

Results from the test are shown in Table 1. These metrics confirm near-linear scaling as the task count was increased (by changing the partition count using repartition). All tests were run on the same cluster with a single 16-core worker node.

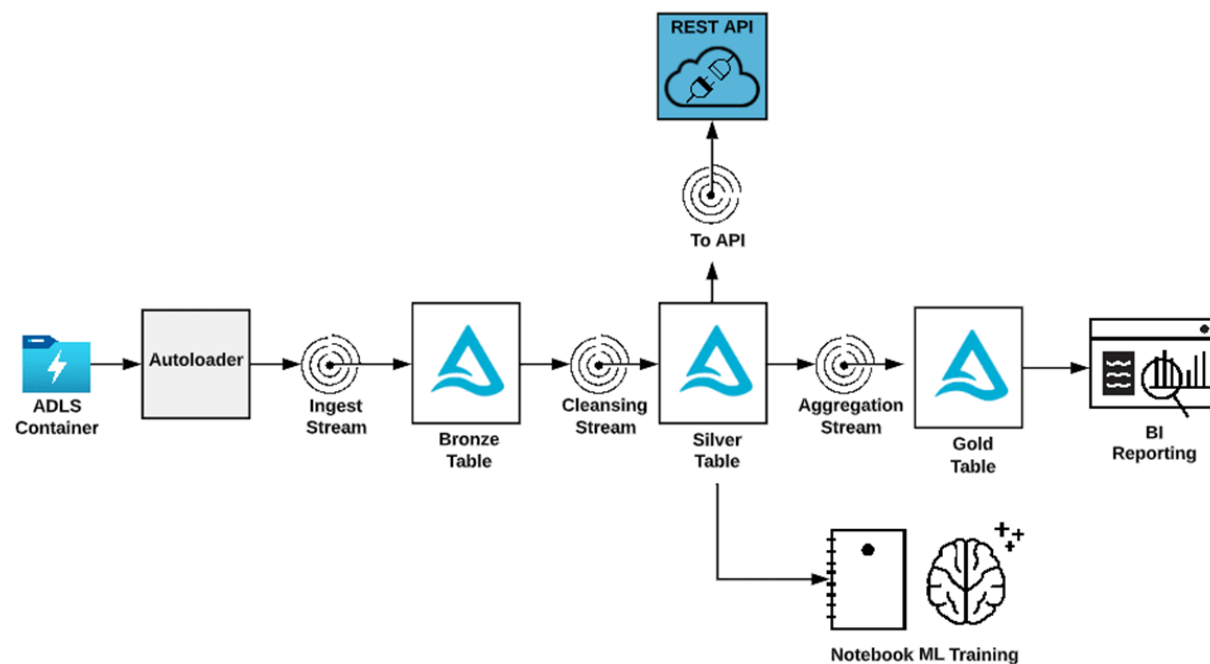
Rows per API Call	Partitions (threads)	Total calls	Seconds to complete processing	Rows per second	Calls per second	Calls per thread per second	Apprx API Latency (ms)
1	4	21,932	179	122	122	30.5	34
	8	21,932	88	249	249	31.1	33
	16	21,932	49	447	447	27.9	38
10	4	21,932	48	456	45	11.3	91
	8	21,932	25	877	87	10.9	100
	16	21,932	13	1687	168	10.5	100

Table 1

REPRESENTATIVE ALL UP PIPELINE DESIGNS

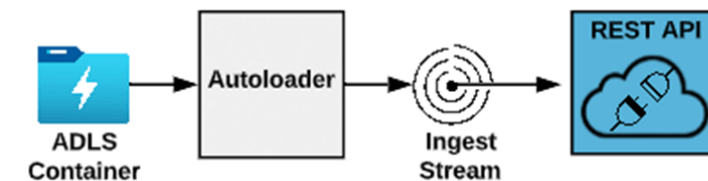
1. Routing some records in a streaming pipeline to REST API (in addition to persistent sinks)

This pattern applies in scenarios where a Spark-based data pipeline already exists for serving analytics or ML use cases. If a requirement emerges to post cleansed or aggregated data to a REST API with low latency, the technique described here can be used.



2. Simple Autoloader to REST API job

This pattern is an example of leveraging the diverse range of sources supported by Structured Streaming. Databricks makes it simple to consume incoming near real-time data – for example using Autoloader to ingest files arriving in cloud storage. Where Databricks is already used for other use cases, this is an easy way to route new streaming sources to a REST API.



SUMMARY

We have shown here how structured streaming can be used to send streamed data to an arbitrary endpoint – in this case, via HTTP POST to a REST API. This opens up many possibilities for flexible integration with analytics data pipelines. However, this is really just one illustration of the power of `foreachBatch` in Spark Structured Streaming.

The `foreachBatch` sink provides the ability to address many endpoint types that are not among the native sinks. Besides REST APIs, these can include databases via JDBC, almost any supported Spark connector, or other cloud services that are addressable via a helper library or API. One example of the latter is pushing data to certain AWS services using the `boto3` library.

This flexibility and scalability enables Structured Streaming to underpin a vast range of real-time solutions across industries.

If you are a Databricks customer, simply follow the [getting started tutorial](#) to familiarize yourself with Structured Streaming. If you are not an existing Databricks customer, [sign up for a free trial](#).

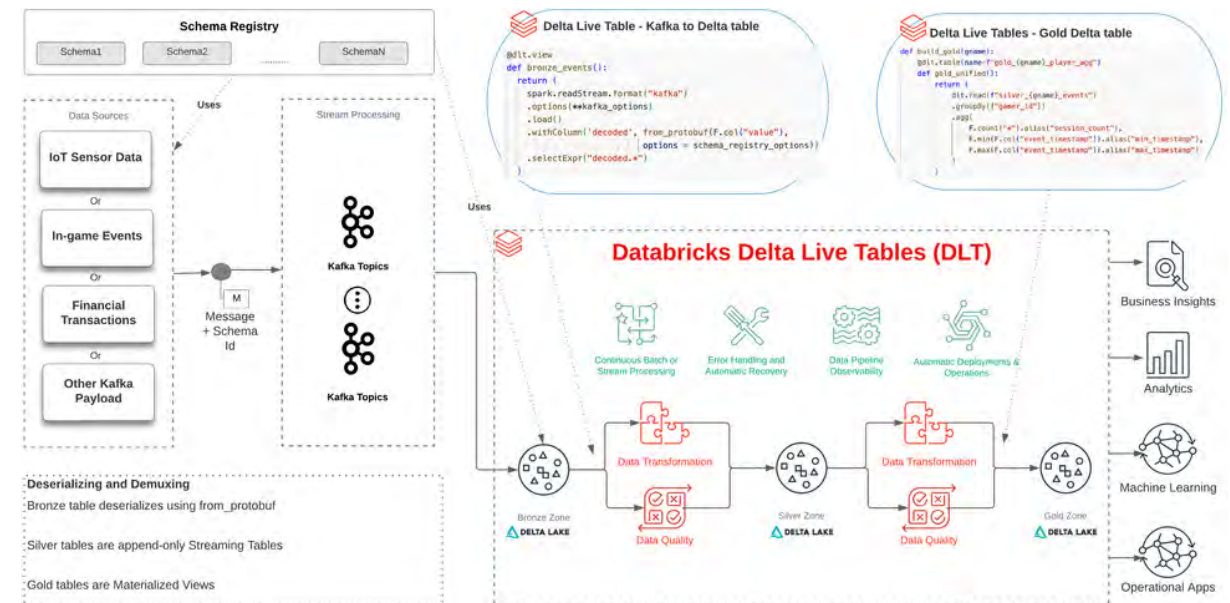
A Data Engineer's Guide to Optimized Streaming With Protobuf and Delta Live Tables

by Craig Lukasik

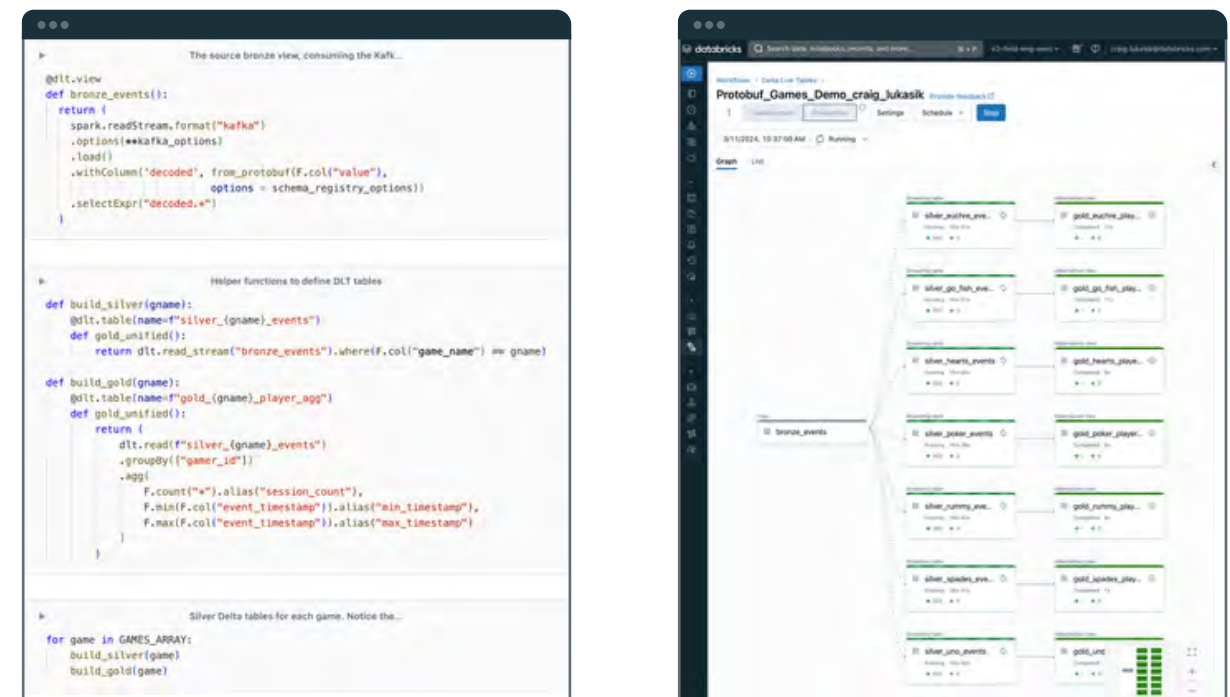
This article describes an example use case where events from multiple games stream through Kafka and terminate in **Delta** tables. The example illustrates how to use **Delta Live Tables** (DLT) to:

1. Stream from **Kafka** into a Bronze Delta table.
2. Consume streaming Protobuf messages with schemas managed by the **Confluent Schema Registry**, handling schema evolution gracefully.
3. **Demultiplex** (demux) messages into multiple game-specific, append-only Silver **Streaming Tables**. Demux indicates that a single stream is split or fanned out into separate streams.
4. Create **Materialized Views** to recalculate aggregate values periodically.

A high-level view of the system architecture is illustrated below.



First, let's look at the Delta Live Tables code for the example and the related pipeline DAG so that we can get a glimpse of the simplicity and power of the DLT framework.



On the left, we see the DLT Python code. On the right, we see the view and the tables created by the code. The bottom cell of the notebook on the left operates on a list of games (GAMES_ARRAY) to dynamically generate the fourteen target tables we see in the DAG.

Before we go deeper into the example code, let’s take a step back and review streaming use cases and some streaming payload format options.

STREAMING OVERVIEW

Skip this section if you’re familiar with streaming use cases, protobuf, the schema registry, and Delta Live Tables. In this article, we’ll dive into a range of exciting topics.

- **Common streaming use cases:** Uncover the diverse streaming data applications in today’s tech landscape.
- **Protocol buffers (Protobuf):** Learn why this fast and compact serialization format is a game-changer for data handling.
- **Delta Live Tables (DLT):** Discover how DLT pipelines offer a rich, feature-packed platform for your **ETL (Extract, Transform, Load)** needs.
- **Building a DLT pipeline:** A step-by-step guide on creating a DLT pipeline that seamlessly consumes Protobuf values from an Apache Kafka stream.
- **Utilizing the Confluent Schema Registry:** Understand how this tool is crucial for decoding binary message payloads effectively.
- **Schema evolution in DLT pipelines:** Navigate the complexities of schema evolution within the DLT pipeline framework when streaming protobuf messages with evolving schema.

COMMON STREAMING USE CASES

The **Databricks Data Intelligence Platform** is a comprehensive data-to-AI enterprise solution that combines data engineers, analysts, and data scientists on a single platform. Streaming workloads can power near real-time prescriptive and **predictive analytics** and automatically retrain Machine Learning (ML) models using **Databricks built-in MLOps support**. The models can be exposed as **scalable, serverless REST** endpoints, all within the Databricks platform.

The data comprising these streaming workloads may originate from various use cases:

STREAMING DATA	USE CASE
IoT sensors on manufacturing floor equipment	Generating predictive maintenance alerts and preemptive part ordering
Set-top box telemetry	Detecting network instability and dispatching service crews
Player metrics in a game	Calculating leader-board metrics and detecting cheat

Data in these scenarios is typically streamed through open source messaging systems, which manage the data transfer from producers to consumers. **Apache Kafka** stands out as a popular choice for handling such payloads. Confluent Kafka and AWS MSK provide robust Kafka solutions for those seeking managed services.

OPTIMIZING THE STREAMING PAYLOAD FORMAT

Databricks provides capabilities that help optimize the AI journey by unifying Business Analysis, Data Science, and Data Analysis activities in a single, governed platform. In your quest to optimize the end-to-end technology stack, a key focus is the **serialization** format of the message payload. This element is crucial for efficiency and performance. We'll specifically explore an optimized format developed by Google, known as **protocol buffers (or "protobuf")**, to understand how it enhances the technology stack.

WHAT MAKES PROTOBUF AN OPTIMIZED SERIALIZATION FORMAT?

Google enumerates the **advantages of protocol buffers**, including compact data storage, fast parsing, availability in many programming languages, and optimized functionality through automatically generated classes.

A key aspect of optimization usually involves using pre-compiled classes in the consumer and producer programs that a developer typically writes. In a nutshell, consumer and producer programs that leverage protobuf are "aware" of a message schema, and the binary payload of a protobuf message benefits from primitive data types and positioning within the binary message, removing the need for field markers or delimiters.

WHY IS PROTOBUF USUALLY PAINFUL TO WORK WITH?

Programs that leverage protobuf must work with classes or modules compiled using protoc (the protobuf compiler). The **protoc compiler** compiles those definitions into classes in various languages, including Java and Python. To learn more about how protocol buffers work, go [here](#).

DATABRICKS MAKES WORKING WITH PROTOBUF EASY

Starting in Databricks Runtime 12.1, Databricks provides native support for **serialization and deserialization between Apache Spark struct....** Protobuf support is implemented as an **Apache Spark DataFrame transformation** and can be used with **Structured Streaming** or for batch operations. It optionally integrates with the **Confluent Schema Registry** (a Databricks-exclusive feature).

Databricks makes it easy to work with protobuf because it handles the protobuf compilation under the hood for the developer. For instance, the data pipeline developer does not have to worry about installing protoc or using it to compile protocol definitions into Python classes.

EXPLORING PAYLOAD FORMATS FOR STREAMING IOT DATA

Before we proceed, it is worth mentioning that JSON or Avro may be suitable alternatives for streaming payloads. These formats offer benefits that, for some use cases, may outweigh protobuf. Let's quickly review these formats.

JSON

JSON is an excellent format for development because it is primarily human-readable. The other formats we'll explore are binary formats, which require tools to inspect the underlying data values. Unlike Avro and protobuf, however, the JSON document is stored as a large string (potentially compressed), meaning more bytes may be used than a value represents. Consider the short int value of 8. A short int requires two bytes. In JSON, you may have a document that looks like the following, and it will require several bytes (~30) for the associated key, quotes, etc.

```
1 {  
2   "my_short": 8  
3 }
```

When we consider protobuf, we expect 2 bytes plus a few more for the overhead related to the positioning metadata.

JSON SUPPORT IN DATABRICKS

On the positive side, JSON documents have rich benefits when used with Databricks. **Databricks Autoloader** can easily transform JSON to a structured DataFrame while also providing built-in support for:

- **Schema inference** – when reading JSON into a DataFrame, you can supply a schema so that the target DataFrame or Delta table has the desired schema. Or you can let the engine **infer the schema**. Alternatively, schema **hints** can be supplied if you want a balance of those features.
- **Schema evolution** – Autoloader provides options for how a workload should adapt to changes in the schema of incoming files.

Consuming and processing JSON in Databricks is simple. To create a Spark DataFrame from JSON files can be as simple as this:

```
1 df = spark.read.format("json").load("example.json")
```

AVRO

Avro is an attractive serialization format because it is compact, encompasses schema information in the files themselves, and has built-in database **support in Databricks that includes schema registry integration**. **This tutorial**, co-authored by Databricks' Angela Chu, walks you through an example that leverages Confluent's Kafka and Schema Registry.

To explore an Avro-based dataset, it is as simple as working with JSON:

```
1 df = spark.read.format("avro").load("example.avro")
```

This datageeks.com article compares Avro and protobuf. It is worth a read if you are on the fence between Avro and protobuf. It describes protobuf as the "fastest amongst all," so if speed outweighs other considerations, such as JSON and Avro's greater simplicity, protobuf may be the best choice for your use case.

EXAMPLE DEMUX PIPELINE

The source code for the end-to-end example is located **on GitHub**. The example includes a simulator (**Producer**), a notebook to install the Delta Live Tables pipeline (**Install_DLT_Pipeline**), and a Python notebook to process the data that is streaming through Kafka (**DLT**).

SCENARIO

Imagine a scenario where a video gaming company is streaming events from game consoles and phone-based games for a number of the games in its portfolio. Imagine the game event messages have a single schema that evolves (i.e., new fields are periodically added). Lastly, imagine that analysts want the data for each game to land in its own **Delta Lake** table. Some analysts and BI tools need pre-aggregated data, too.

Using DLT, our pipeline will create 1+2N tables:

- One table for the raw data (stored in the Bronze view).
- One Silver Streaming Table for each of the N games, with events streaming through the Bronze table.
- Each game will also have a Gold Delta table with aggregates based on the associated Silver table.

CODE WALKTHROUGH

BRONZE TABLE DEFINITION

We'll define the Bronze table (bronze_events) as a **DLT view** by using the `@dlt.view` annotation

```
1 import pyspark.sql.functions as F
2 from pyspark.sql.protobuf.functions import from_protobuf
3
4 @dlt.view
5 def bronze_events():
6     return (
7         spark.readStream.format("kafka")
8         .options(**kafka_options)
9         .load()
10        .withColumn('decoded', from_protobuf(F.col("value"), options = schema_registry_
11        options))
12        .selectExpr("decoded.*")
13    )
```

The repo includes the source code that constructs values for `kafka_options`. These details are needed so the streaming Delta Live Table can consume messages from the Kafka topic and retrieve the schema from the Confluent Schema registry (via config values in `schema_registry_options`). This line of code is what manages the deserialization of the protobuf messages:

```
1 .withColumn('decoded', from_protobuf(F.col("value"), options = schema_registry_
2 options))
```

The simplicity of transforming a DataFrame with protobuf payload is thanks to this function: `from_protobuf` (available in Databricks Runtime 12.1 and later). In this article, we don't cover `to_protobuf`, but the ease of use is the same. The `schema_registry_options` are used by the function to look up the schema from the Confluent Schema Registry.

Delta Live Tables is a declarative ETL framework that simplifies the development of data pipelines. So, suppose you are familiar with **Apache Spark Structured Streaming**. In that case, you may notice the absence of a `checkpointLocation` (which is required to track the stream's progress so that the stream can be stopped and started without duplicating or dropping data). The absence of the `checkpointLocation` is because Delta Live Tables manages this need out-of-the-box for you. Delta Live Tables also has other features that help make developers more agile and provide a common framework for ETL across the enterprise. **Delta Live Tables Expectations**, used for managing data quality, is one such feature.

SILVER TABLES

The following function creates a Silver Streaming Table for the given game name provided as a parameter:

```
1 def build_silver(gname):
2     .table(name=f"silver_{gname}_events")
3     def gold_unified():
4         return dlt.read_stream("bronze_events").where(F.col("game_name") == gname)
```

Notice the use of the `@dlt.table` annotation. Thanks to this annotation, when `build_silver` is invoked for a given `gname`, a DLT table will be defined that depends on the source `bronze_events` table. We know that the tables created by this function will be Streaming Tables because of the use of `dlt.read_stream`.

GOLD TABLES

The following function creates a Gold Materialized View for the given game name provided as a parameter:

```
1 def build_gold(gname):
2     .table(name=f"gold_{gname}_player_agg")
3     def gold_unified():
4         return (
5             dlt.read(f"silver_{gname}_events")
6             .groupBy(["gamer_id"])
7             .agg(
8                 F.count("*").alias("session_count"),
9                 F.min(F.col("event_timestamp")).alias("min_timestamp"),
10                F.max(F.col("event_timestamp")).alias("max_timestamp")
11            )
12        )
```

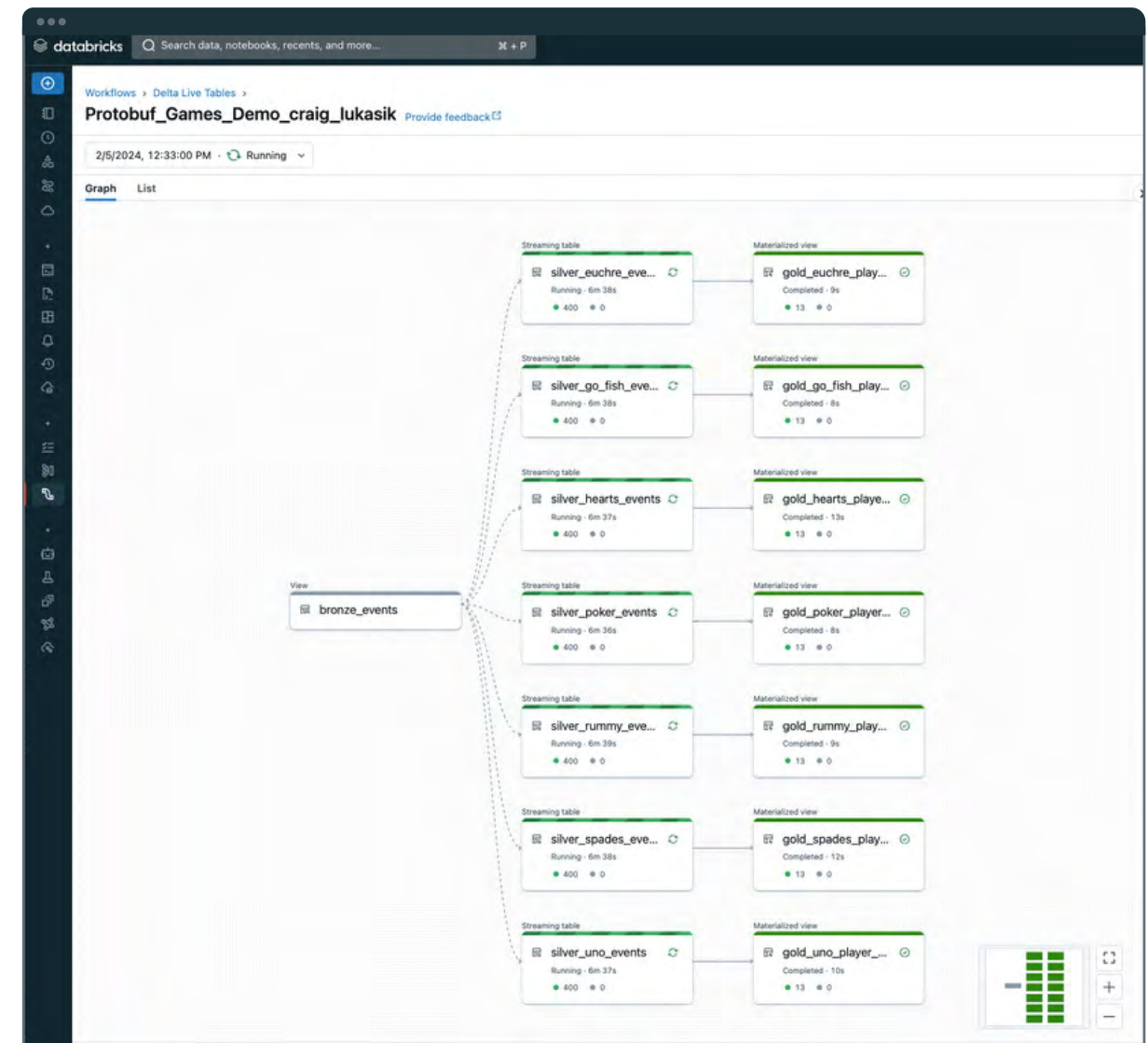
We know the resulting table will be a "Materialized View" because of the use of `dlt.read`. This is a simple Materialized View definition; it simply performs a count of source events along with min and max event times, grouped by `gamer_id`.

METADATA-DRIVEN TABLES

The previous two sections of this article defined functions for creating Silver (Streaming) Tables and Gold Materialized Views. The metadata-driven approach in the example code uses a pipeline input parameter to create $N \times 2$ target tables (one Silver table for each game and one aggregate Gold table for each game). This code drives the dynamic table creation using the aforementioned `build_silver` and `build_gold` functions:

```
1 GAMES_ARRAY = spark.conf.get("games").split(",")
2 for game in GAMES_ARRAY:
3     build_silver(game)
4     build_gold(game)
```

At this point, you might have noticed that much of the control flow code data engineers often have to write is absent. This is because, as mentioned above, DLT is a declarative programming framework. It automatically detects dependencies and manages the pipeline's execution flow. Here's the DAG that DLT creates for the pipeline:



A note about aggregates in a streaming pipeline

For a continuously running stream, calculating some aggregates can be very resource-intensive. Consider a scenario where you must calculate the "median" for a continuous stream of numbers. Every time a new number arrives in the stream, the median calculation will need to explore the entire set of numbers that have ever arrived. In a stream receiving millions of numbers per second, this fact can present a significant challenge if your goal is to provide a destination table for the median of the entire stream of numbers. It becomes impractical to perform such a feat every time a new number arrives. The limits of computation power and persistent storage and network would mean that the stream would continue to grow a backlog much faster than it could perform the calculations.

In a nutshell, it would not work out well if you had such a stream and tried to recalculate the median for the universe of numbers that have ever arrived in the stream. So, what can you do? If you look at the code snippet above, you may notice that this problem is not addressed in the code! Fortunately, as a Delta Live Tables developer, I do not have to worry about it. The declarative framework handles this dilemma by design. DLT addresses this by materializing results only periodically. Furthermore, DLT provides a table property that allows the developer to set an appropriate **trigger interval**.

REVIEWING THE BENEFITS OF DLT

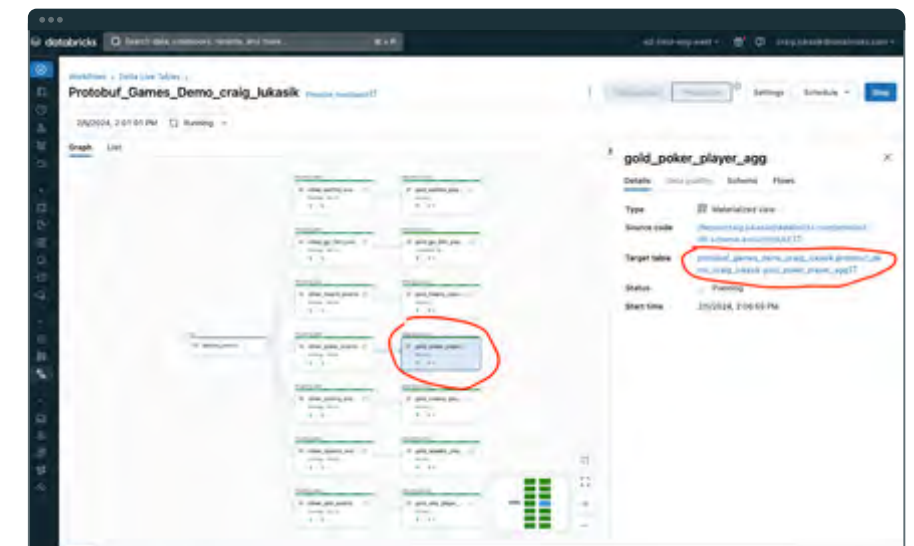
Governance

Unity Catalog governs the end-to-end pipeline. Thus, permission to target tables can be granted to end-users and service principals needing access across any Databricks workspaces attached to the same **metastore**.

Lineage

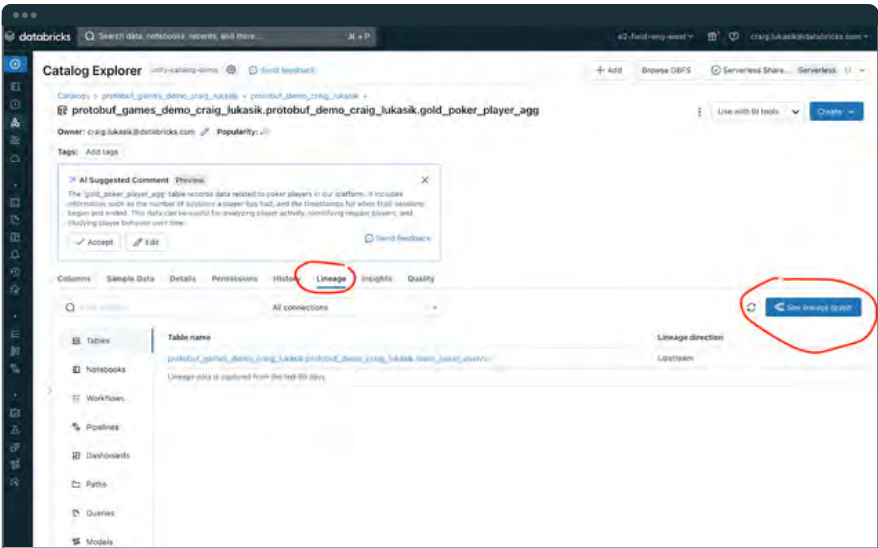
From the Delta Live Tables interface, we can navigate to the Catalog and view lineage.

Click on a table in the DAG. Then click on the "Target table" link.

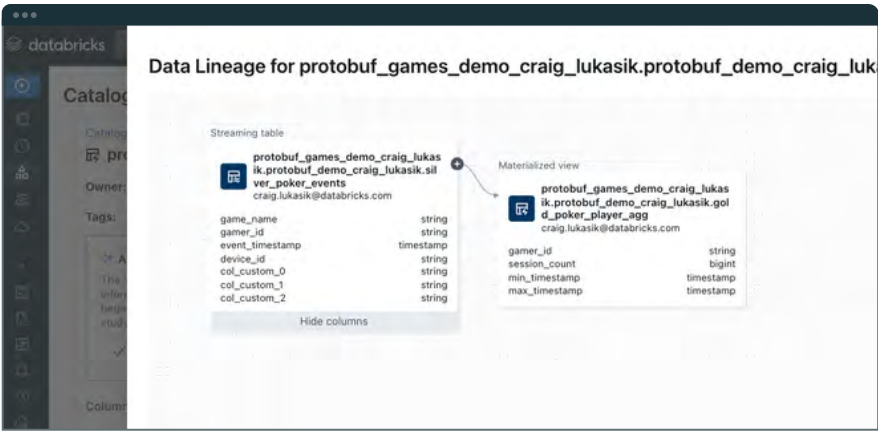


Click on the "Lineage" tab for the table. Then click on the "See lineage graph" link.

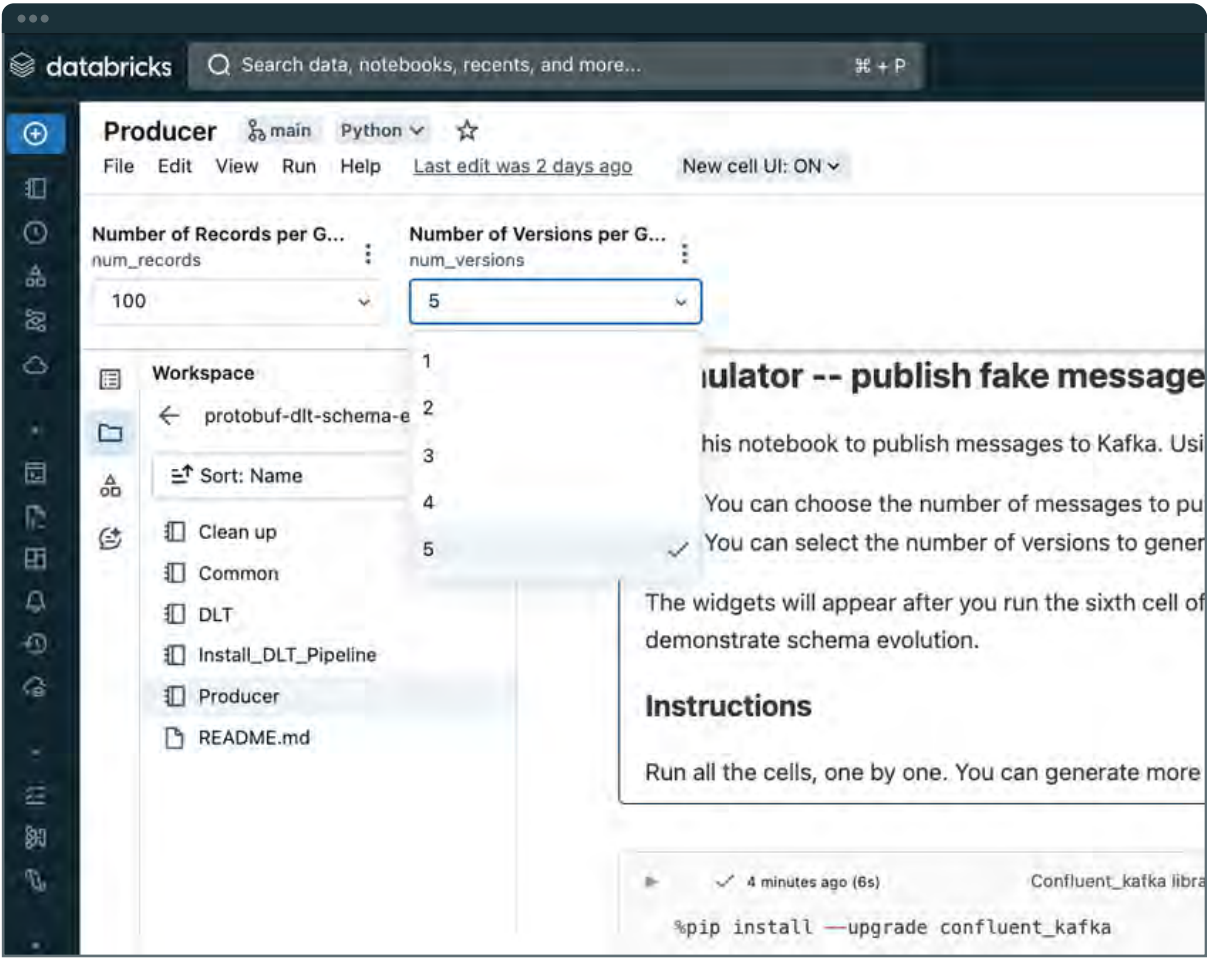
Lineage also provides visibility into other related artifacts, such as notebooks, models, etc.



This lineage helps accelerate team velocity by making it easier to understand how assets in the workspace are related.

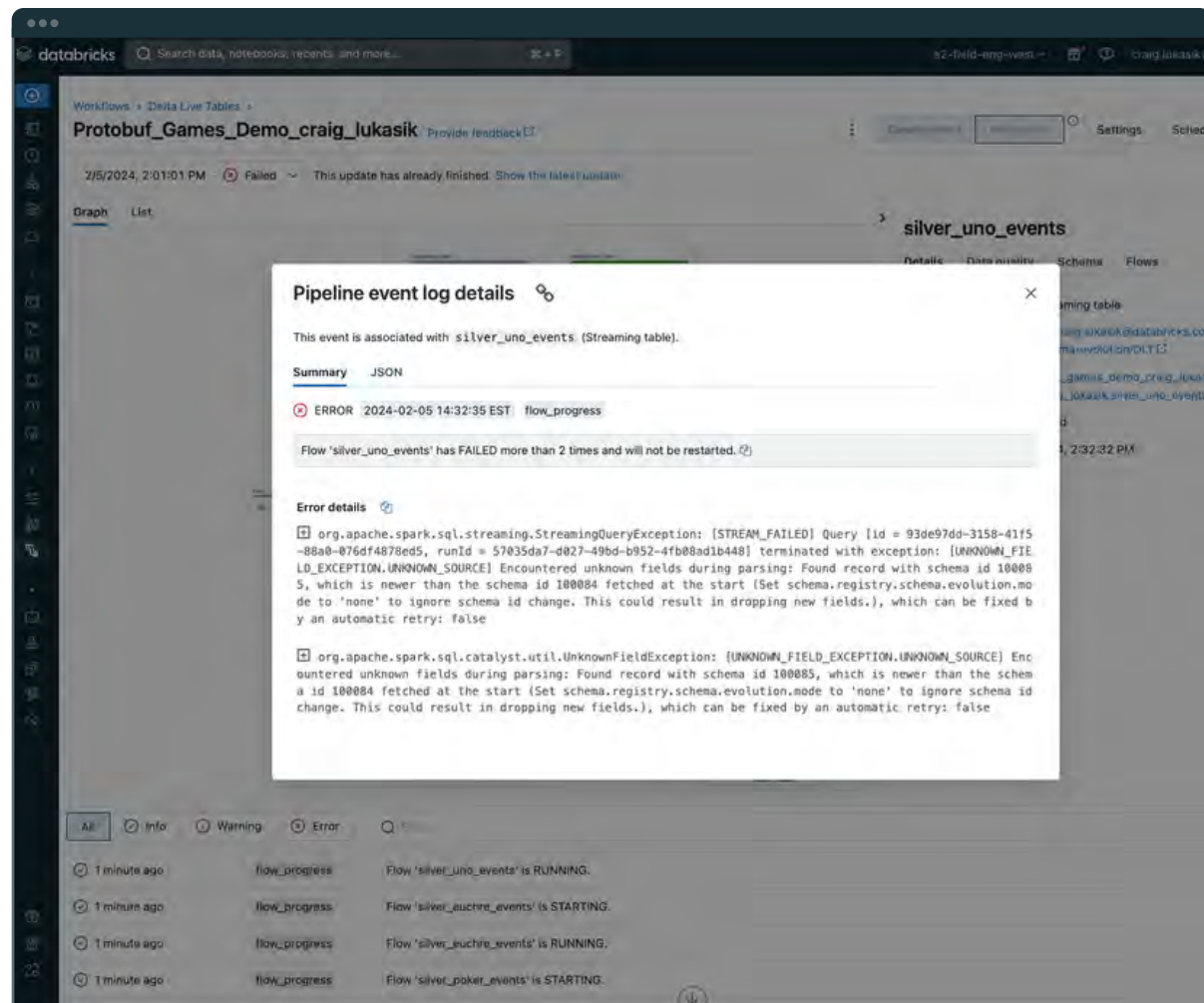


HANDS-OFF SCHEMA EVOLUTION

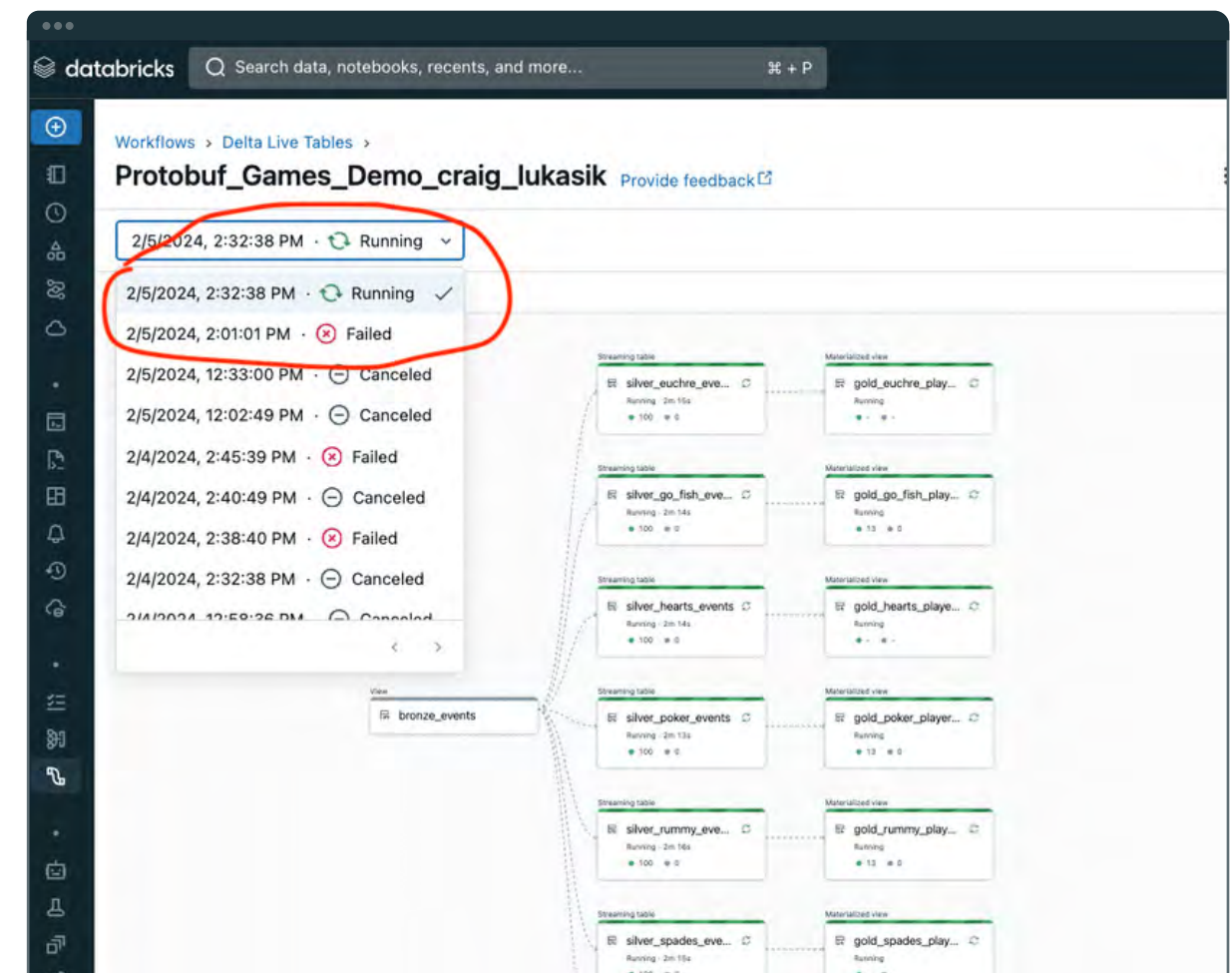


Delta Live Tables will detect this as the source stream's schema evolves, and the pipeline will restart. To simulate a schema evolution for this example, you would run the Producer notebook a subsequent time but with a larger value for num_versions, as shown on the left. This will generate new data where the schema includes some additional columns. The Producer notebook updates the schema details in the Confluent Schema Registry.

When the schema evolves, you will see a pipeline failure like this one:



If the Delta Live Tables pipeline runs in **Production mode**, a failure will result in an automatic pipeline restart. The Schema Registry will be contacted upon restart to retrieve the latest schema definitions. Once back up, the stream will continue with a new run:



CONCLUSION

In high-performance IoT systems, optimization extends through every layer of the technology stack, focusing on the payload format of messages in transit. Throughout this article, we've delved into the benefits of using an optimized serialization format, protobuf, and demonstrated its integration with Databricks to construct a comprehensive end-to-end demultiplexing pipeline. This approach underlines the importance of selecting the right tools and formats to maximize efficiency and effectiveness in IoT systems.

Instructions for running the example

To run the example code, follow these instructions:

1. In Databricks, clone this repo:
<https://github.com/craig-db/protobuf-dlt-schema-evolution>.
2. Set up the prerequisites (documented below).
3. Follow the instructions in the README notebook included in the repo code.

Prerequisites

1. A Unity Catalog-enabled workspace — this demo uses a Unity Catalog-enabled Delta Live Tables pipeline. Thus, Unity Catalog should be configured for the workspace where you plan to run the demo.
2. As of January 2024, you should use the Preview channel for the Delta Live Tables pipeline. The "Install_DLT_Pipeline" notebook will use the Preview channel when installing the pipeline.
3. Confluent account — this demo uses Confluent Schema Registry and Confluent Kafka.

Secrets to configure

The following Kafka and Schema Registry connection details (and credentials) should be saved as Databricks Secrets and then set within the Secrets notebook that is part of the repo:

- **SR_URL:** Schema Registry URL
(e.g. <https://myschemaregistry.aws.confluent.cloud>)
- **SR_API_KEY:** Schema Registry API Key
- **SR_API_SECRET:** Schema Registry API Secret
- **KAFKA_KEY:** Kafka API Key
- **KAFKA_SECRET:** Kafka Secret
- **KAFKA_SERVER:** Kafka host:port (e.g. mykafka.aws.confluent.cloud:9092)
- **KAFKA_TOPIC:** The Kafka Topic
- **TARGET_SCHEMA:** The target database where the streaming data will be appended into a Delta table (the destination table is named unified_gold)
- **CHECKPOINT_LOCATION:** Some location (e.g., in **DBFS**) where the **checkpoint** data for the streams will be stored

Go here to learn how to save secrets to secure sensitive information (e.g., credentials) within the Databricks Workspace: <https://docs.databricks.com/security/secrets/index.html>.

Design Patterns for Batch Processing in Financial Services

by Eon Retief

Financial services institutions (FSIs) around the world are facing unprecedented challenges ranging from market volatility and political uncertainty to changing legislation and regulations. Businesses are forced to accelerate digital transformation programs; automating critical processes to reduce operating costs and improve response times. However, with data typically scattered across multiple systems, accessing the information required to execute on these initiatives tends to be easier said than done.

Architecting an ecosystem of services able to support the plethora of data-driven use cases in this digitally transformed business can, however, seem to be an impossible task. This chapter will focus on one crucial aspect of the modern data stack: batch processing. A seemingly outdated paradigm, we'll see why batch processing remains a vital and highly viable component of the data architecture. And we'll see how Databricks can help FSIs navigate some of the crucial challenges faced when building infrastructure to support these scheduled or periodic workflows.

WHY BATCH INGESTION MATTERS

Over the last two decades, the global shift towards an instant society has forced organizations to rethink the operating and engagement model. A digital-first strategy is no longer optional but vital for survival. Customer needs and demands are changing and evolving faster than ever. This desire for instant gratification is driving an increased focus on building capabilities that support real-time processing and decisioning. One might ask whether batch processing is still relevant in this new dynamic world.

While real-time systems and streaming services can help FSIs remain agile in addressing the volatile market conditions at the edge, they do not typically meet the requirements of back-office functions. Most business decisions are not reactive but rather, require considered, strategic reasoning. By definition, this approach requires a systematic review of aggregate data collected over a period of time. Batch processing in this context still provides the most efficient and cost-effective method for processing large, aggregate volumes of data. Additionally, batch processing can be done offline, reducing operating costs and providing greater control over the end-to-end process.

The world of finance is changing, but across the board incumbents and startups continue to rely heavily on batch processing to power core business functions. Whether for reporting and risk management or anomaly detection and surveillance, FSIs require batch processing to reduce human error, increase the speed of delivery, and reduce operating costs.

GETTING STARTED

Starting with a 30,000-ft view, most FSIs will have a multitude of data sources scattered across on-premises systems, cloud-based services and even third-party applications. Building a batch ingestion framework that caters for all these connections require complex engineering and can quickly become a burden on maintenance teams. And that's even before considering things like change data capture (CDC), scheduling, and schema evolution. In this section, we will demonstrate how the Databricks **Lakehouse for Financial Services** (LFS) and its ecosystem of partners can be leveraged to address these key challenges and greatly simplify the overall architecture.

The Databricks lakehouse architecture was designed to provide a unified platform that supports all analytical and scientific data workloads. Figure 1 shows the reference architecture for a decoupled design that allows easy integration with other platforms that support the modern data ecosystem. The lakehouse makes it easy to construct ingestion and serving layers that operate irrespective of the data's source, volume, velocity, and destination.

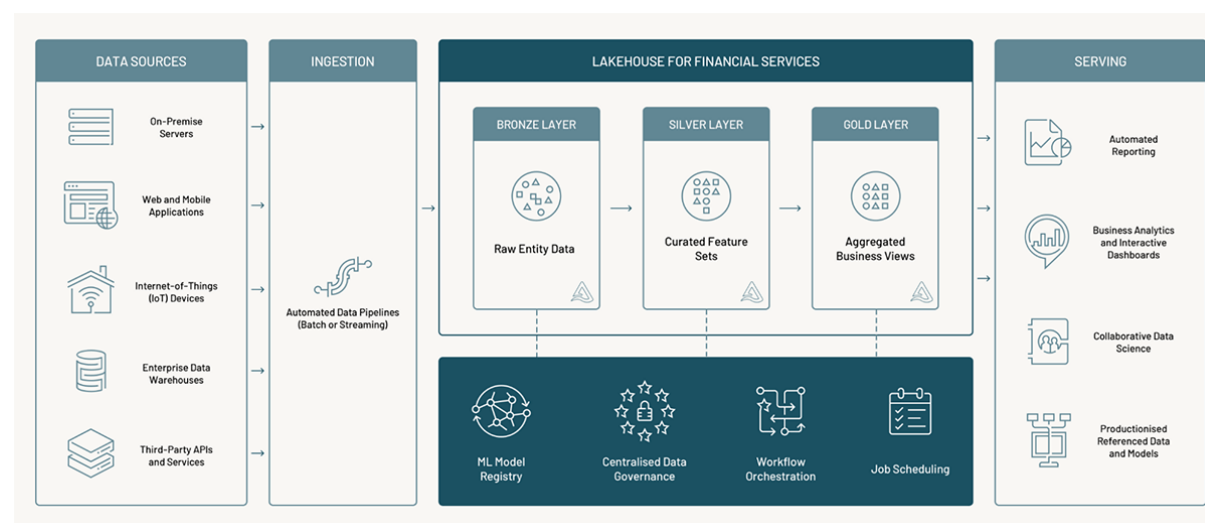


Figure 1: Reference architecture of the Lakehouse for Financial Services

To demonstrate the power and efficiency of the LFS, we turn to the world of insurance. We consider the basic reporting requirements for a typical claims workflow. In this scenario, the organization might be interested in the key metrics driven by claims processes. For example:

- Number of active policies
- Number of claims
- Value of claims
- Total exposure
- Loss ratio

Additionally, the business might want a view of potentially suspicious claims and a breakdown by incident type and severity. All these metrics are easily calculable from two key sources of data: 1) the book of policies and 2) claims filed by customers. The policy and claims records are typically stored in a combination of enterprise data warehouses (EDWs) and operational databases. The main challenge becomes connecting to these sources and ingesting data into our lakehouse, where we can leverage the power of Databricks to calculate the desired outputs.

Luckily, the flexible design of the LFS makes it easy to leverage best-in-class products from a range of SaaS technologies and tools to handle specific tasks. One possible solution for our claims analytics use case would be to use Fivetran for the batch ingestion plane. Fivetran provides a simple and secure platform for connecting to numerous data sources and delivering data directly to the Databricks lakehouse. Additionally, it offers native support for CDC, schema evolution and workload scheduling. In Figure 2, we show the technical architecture of a practical solution for this use case.

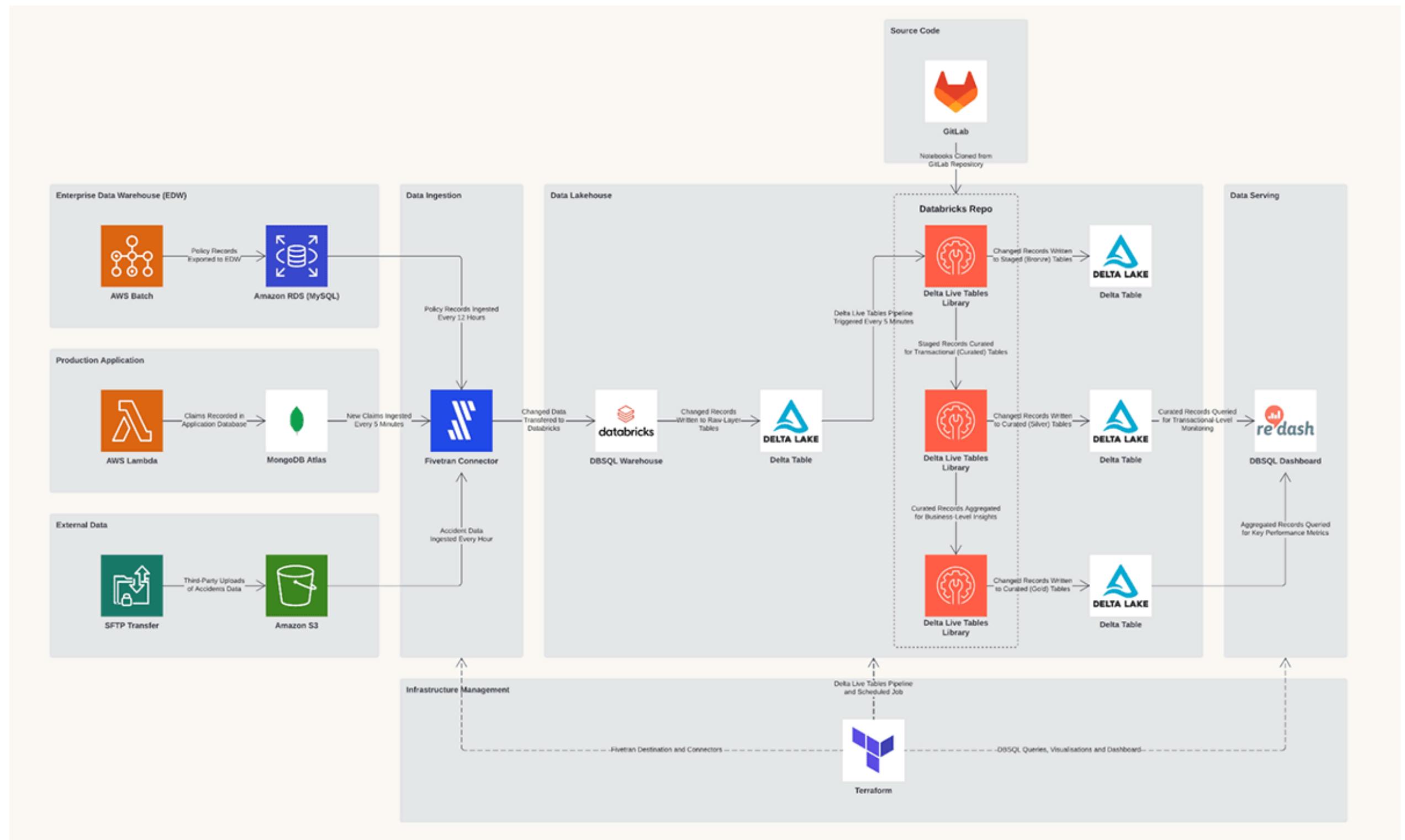


Figure 2: Technical architecture for a simple insurance claims workflow

Once the data is ingested and delivered to the LFS, we can use **Delta Live Tables (DLT)** for the entire engineering workflow. DLT provides a simple, scalable declarative framework for automating complex workflows and enforcing data quality controls. The outputs from our DLT workflow, our curated and aggregated assets, can be interrogated using **Databricks SQL (DB SQL)**. DB SQL brings data warehousing to the LFS to power business-critical analytical workloads. Results from DB SQL queries can be packaged in easy-to-consume dashboards and served to business users.

STEP 1: CREATING THE INGESTION LAYER

Setting up an ingestion layer with Fivetran requires a two-step process. First, configuring a so-called *destination* where data will be delivered, and second, establishing one or more connections with the source systems. The **Partner Connect** interface takes care of the first step with a simple, guided interface to connect Fivetran with a Databricks Warehouse. Fivetran will use the warehouse to convert raw source data to Delta Tables and store the results in the Databricks Lakehouse. Figures 3 and 4 show steps from the Partner Connect and Fivetran interfaces to configure a new destination.

The screenshot shows a web interface titled "Connect to partner" with a close button (X) in the top right corner. The Fivetran logo is displayed below the title. A message states: "Databricks has created resources to connect with Fivetran. To sign in to your Fivetran account, click Connect to Fivetran." Below this, there are several input fields for configuration:

- Email**: A text input field.
- Connection details**: A dropdown menu.
- User**: A text input field containing "FIVETRAN_USER" and a copy icon.
- Personal access token**: A text input field containing "*****" and a copy icon.
- Server Hostname**: A text input field containing ".cloud.databricks.com" and a copy icon.
- Port**: A text input field containing "443" and a copy icon.
- HTTP path**: A text input field containing "/sql/1.0/warehouses/" and a copy icon.

At the bottom, there is a blue button labeled "Connect to Fivetran" and a white button labeled "Cancel". A disclaimer box at the bottom states: "By clicking Connect to Fivetran, you are instructing Databricks to provide all of the above information in addition to your name and email address to Fivetran. Fivetran's processing of this information and your use of Fivetran's products and services are governed solely by Fivetran's [terms and conditions](#) and Fivetran's [privacy policy](#) and not your agreement with Databricks. Please contact Fivetran support if you have questions about connecting with Fivetran prior to continuing."

Figure 3: Databricks Partner Connect interface for creating a new connection

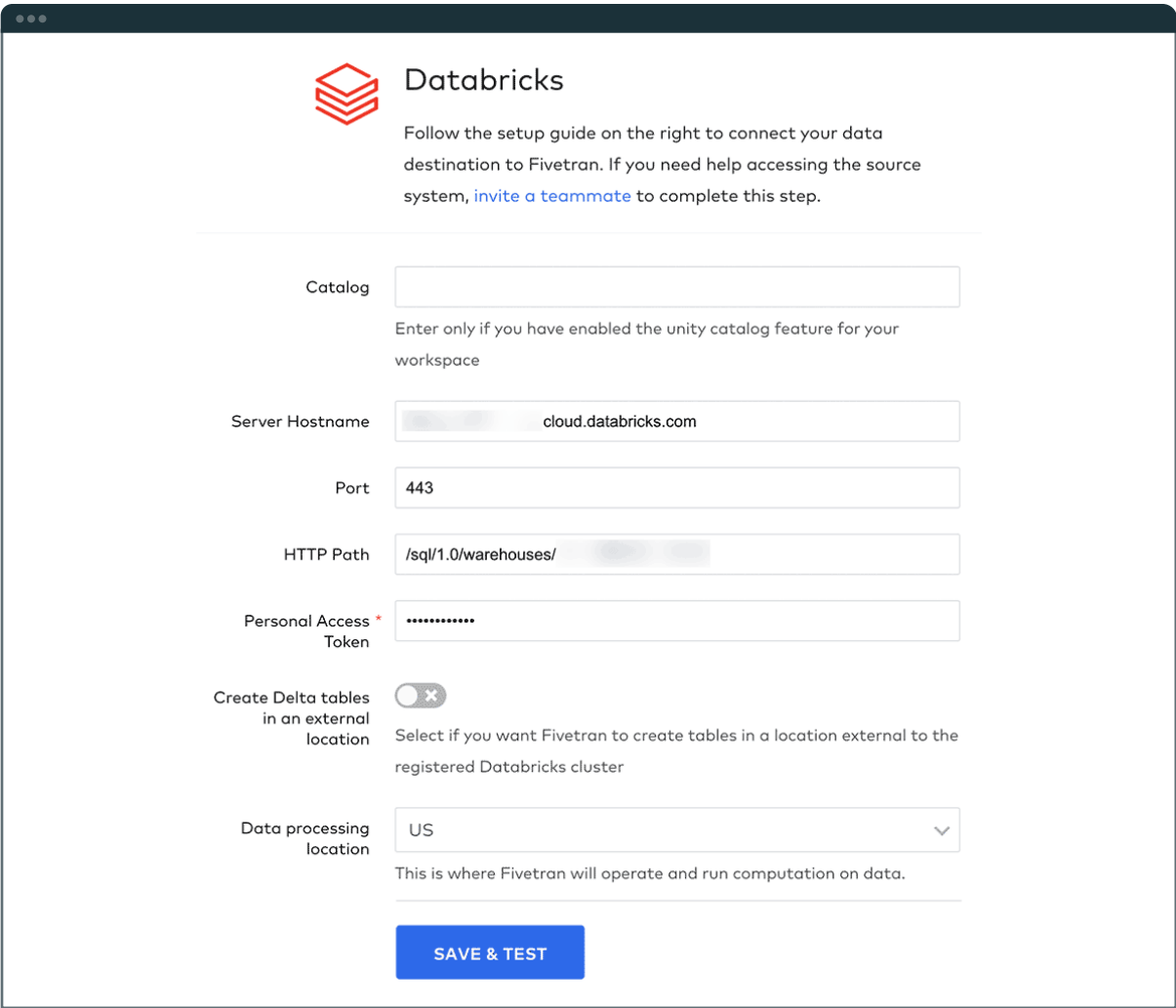


Figure 4: Fivetran interface for confirming a new destination

For the next step, we move to the Fivetran interface. From here, we can easily create and configure connections to several different source systems (please refer to the [official documentation](#) for a complete list of all supported connections). In our example, we consider three sources of data: 1) policy records stored in an Operational Data Store (ODS) or Enterprise Data Warehouse (EDW), 2) claims records stored in an operational database, and 3) external data delivered to blob storage. As such, we require three different connections to be configured in Fivetran. For each of these, we can follow Fivetran’s simple guided process to set up a connection with the source system. Figures 5 and 6 show how to configure new connections to data sources.

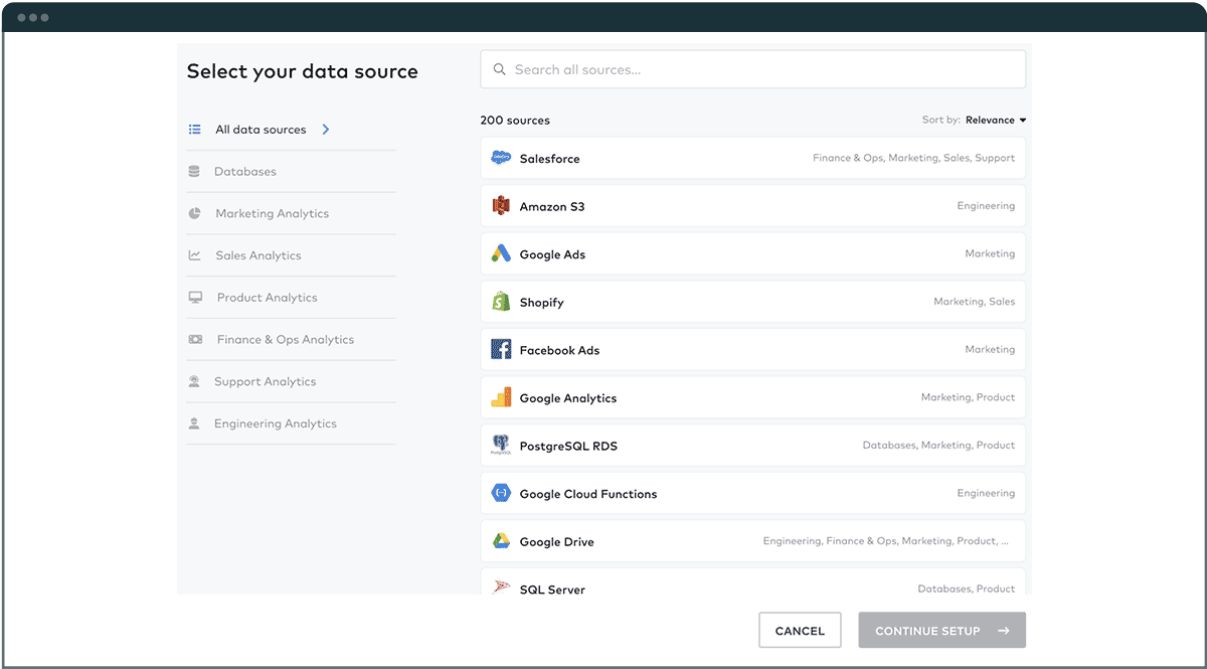


Figure 5: Fivetran interface for selecting a data source type

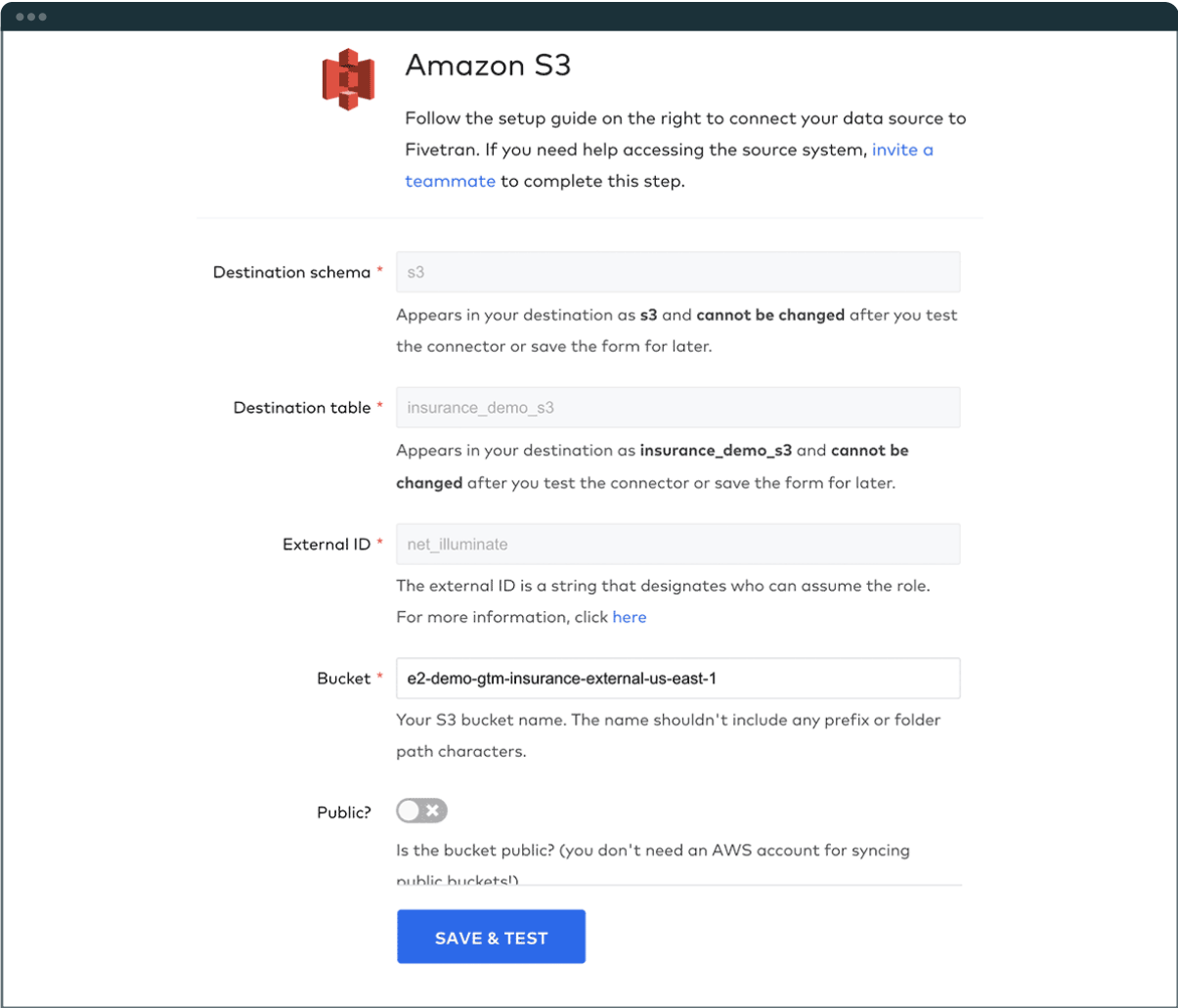


Figure 6: Fivetran interface for configuring a data source connection

Connections can further be configured once they have been validated. One important option to set is the frequency with which Fivetran will interrogate the source system for new data. In Figure 7, we can see how easy Fivetran has made it to set the sync frequency with intervals ranging from 5 minutes to 24 hours.

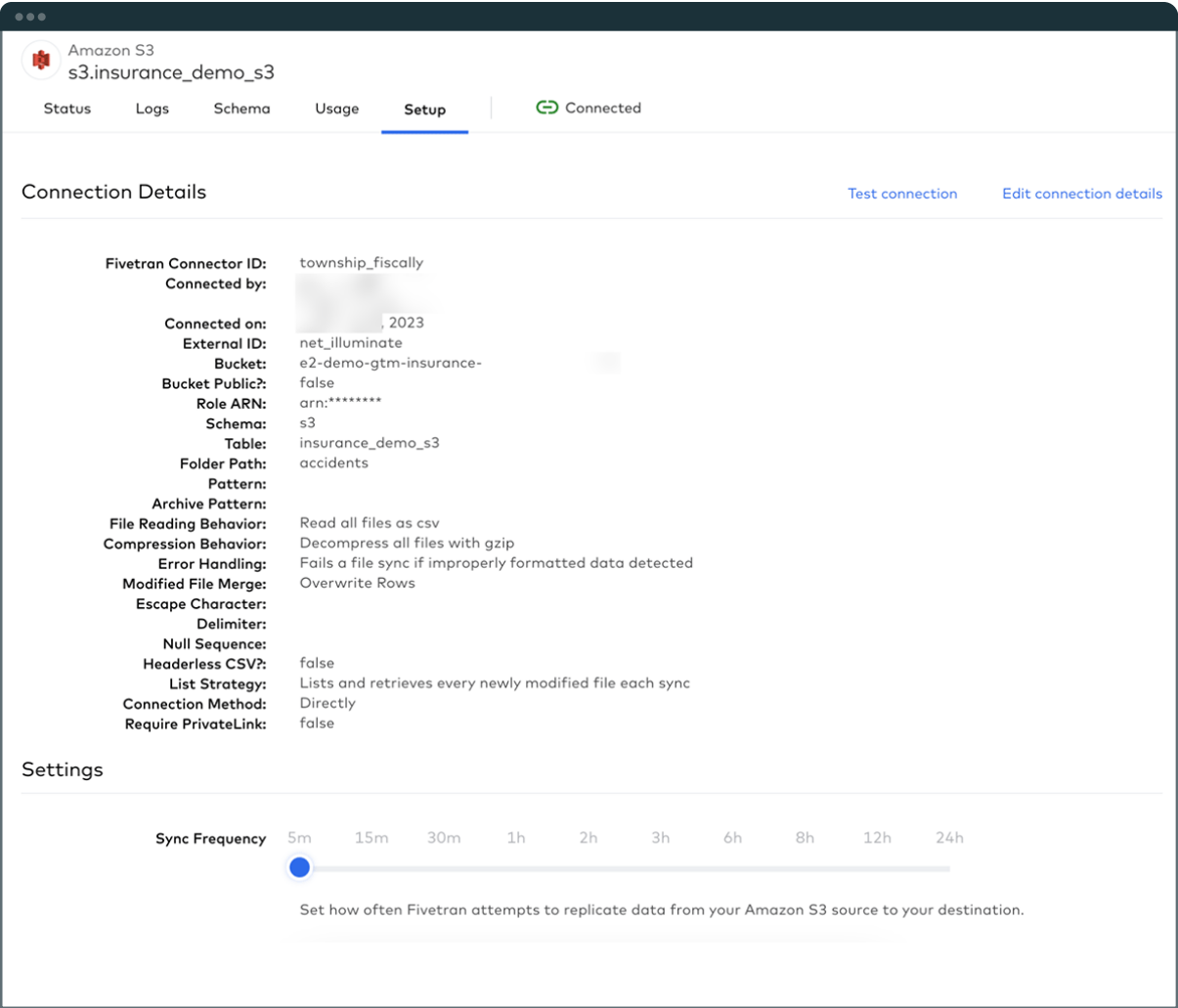


Figure 7: Overview of configuration for a Fivetran connect

Fivetran will immediately interrogate and ingest data from source systems once a connection is validated. Data is stored as Delta tables and can be viewed from within Databricks through the DB SQL **Data Explorer**. By default, Fivetran will store all data under the Hive metastore. A new schema is created for each new connection, and each schema will contain at least two tables: one containing the data and another with logs from each attempted ingestion cycle (see Figure 8).

Catalogs > hive_metastore >

hive_metastore.insurance_demo_mongodb_master

Schema

Upgrade

Tables Details Permissions

2 Tables

Filter tables...

Name	Created at	Owner
claims		
fivetran_audit		

Figure 8: Summary of tables created by Fivetran in the Databricks Warehouse for an example connection

Having the data stored in Delta tables is a significant advantage. Delta Lake natively supports granular data versioning, meaning we can time travel through each ingestion cycle (see Figure 9). We can use DB SQL to interrogate specific versions of the data to analyze how the source records evolved.

Catalogs > hive_metastore > insurance_demo_mongodb_master >

hive_metastore.insurance_demo_mongodb_master.fivetran_audit

Table Delta 6.4KiB, 2 files Add comment

Actions Upgrade

Columns Sample Data Details Permissions History

Version	Timestamp	User Id	User Name	Operation	Operation Parameters	Job	Notebook	Cluster Id	Read Version	Isolation Level	Is Blind Append	Operation Metrics	User Metadata	Engine In
2	2022-06-28T23:06:50	Null	Null	MERGE	{ "matchedPredicates": [{"actionType": "update"}], "notMatchedPredicates": [{"actionType": "insert"}], "predicate": "(main.id = _fivetran_staging_id)" }	Null	Null	Null	1	WriteSerializable	false	{ ... } // 12 items	Null	Databrick Runtime/ photon-scala2.12
1	2022-06-21T14:34:52	Null	Null	COPY INTO	{ } // 0 items	Null	Null	Null	0	WriteSerializable	true	{ ... } // 3 items	Null	Databrick Runtime/ photon-scala2.12
0	2022-06-21T14:34:42	Null	Null	CREATE TABLE	{ ... } // 4 items	Null	Null	Null	Null	WriteSerializable	true	{ ... } // 0 items	Null	Databrick Runtime/ photon-scala2.12

Figure 9: View of the history showing changes made to the Fivetran audit table

It is important to note that if the source data contains semi-structured or unstructured values, those attributes will be flattened during the conversion process. This means that the results will be stored in grouped text-type columns, and these entities will have to be dissected and unpacked with DLT in the curation process to create separate attributes.

STEP 2: AUTOMATING THE WORKFLOW

With the data in the Databricks Data Intelligence Platform, we can use Delta Live Tables (DLT) to build a simple, automated data engineering workflow. DLT provides a declarative framework for specifying detailed feature engineering steps. Currently, DLT supports APIs for both Python and SQL. In this example, we will use Python APIs to build our workflow.

The most fundamental construct in DLT is the definition of a table. DLT interrogates all table definitions to create a comprehensive workflow for how data should be processed. For instance, in Python, tables are created using function definitions and the `@dlt.table` decorator (see example of Python code below). The decorator is used to specify the name of the resulting table, a descriptive comment explaining the purpose of the table, and a collection of table properties.

```

1  @dlt.table(
2      name          = "curated_claims",
3      comment       = "Curated claim records",
4      table_properties = {
5          "layer": "silver",
6          "pipelines.autoOptimize.managed": "true",
7          "delta.autoOptimize.optimizeWrite": "true",
8          "delta.autoOptimize.autoCompact": "true"
9      }
10 )
11 def curate_claims():
12     # Read the staged claim records into memory
13     staged_claims = dlt.read("staged_claims")
14     # Unpack all nested attributes to create a flattened table structure
15     curated_claims = unpack_nested(df = staged_claims, schema = schema_claims)
16     ...

```

Instructions for feature engineering are defined inside the function body using standard PySpark APIs and native Python commands. The following example shows how PySpark joins claims records with data from the policies table to create a single, curated view of claims.

```
1  ...
2
3  # Read the staged claim records into memory
4  curated_policies = dlt.read("curated_policies")
5  # Evaluate the validity of the claim
6  curated_claims = curated_claims \
7      .alias("a") \
8      .join(
9          curated_policies.alias("b"),
10         on = F.col("a.policy_number") == F.col("b.policy_number"),
11         how = "left"
12     ) \
13     .select([F.col(f"a.{c}") for c in curated_claims.columns] + [F.col(f"b.{c}")
14     alias(f"policy_{c}") for c in ("effective_date", "expiry_date")]) \
15     .withColumn(
16         # Calculate the number of months between coverage starting and the
17         claim being filed
18         "months_since_covered", F.round(F.months_between(F.col("claim_date"),
19         F.col("policy_effective_date")))
20     ) \
21     .withColumn(
22         # Check if the claim was filed before the policy came into effect
23         "claim_before_covered", F.when(F.col("claim_date") < F.col("policy_
24         effective_date"), F.lit(1)).otherwise(F.lit(0))
25     ) \
26     .withColumn(
27         # Calculate the number of days between the incident occurring and the
28         claim being filed
29         "days_between_incident_and_claim", F.datediff(F.col("claim_date"),
30         F.col("incident_date"))
31     )
32
33 # Return the curated dataset
34 return curated_claims
```

One significant advantage of DLT is the ability to specify and enforce data quality standards. We can set expectations for each DLT table with detailed data quality constraints that should be applied to the contents of the table. Currently, DLT supports expectations for three different scenarios:

DECORATOR	DESCRIPTION
expect	Retain records that violate expectations
expect_or_drop	Drop records that violate expectations
expect_or_fail	Halt the execution if any record(s) violate constraints

Expectations can be defined with one or more data quality constraints. Each constraint requires a description and a Python or SQL expression to evaluate. Multiple constraints can be defined using the expect_all, expect_all_or_drop, and expect_all_or_fail decorators. Each decorator expects a Python dictionary where the keys are the constraint descriptions, and the values are the respective expressions. The example below shows multiple data quality constraints for the retain and drop scenarios described above.

```

1  @dlt.expect_all({
2      "valid_driver_license": "driver_license_issue_date > (current_date() -
3      cast(cast(driver_age AS INT) AS INTERVAL YEAR))",
4      "valid_claim_amount": "total_claim_amount > 0",
5      "valid_coverage": "months_since_covered > 0",
6      "valid_incident_before_claim": "days_between_incident_and_claim > 0"
7  })
8  @dlt.expect_all_or_drop({
9      "valid_claim_number": "claim_number IS NOT NULL",
10     "valid_policy_number": "policy_number IS NOT NULL",
11     "valid_claim_date": "claim_date < current_date()",
12     "valid_incident_date": "incident_date < current_date()",
13     "valid_incident_hour": "incident_hour between 0 and 24",
14     "valid_driver_age": "driver_age > 16",
15     "valid_effective_date": "policy_effective_date < current_date()",
16     "valid_expiry_date": "policy_expiry_date <= current_date()"
17 })
18 }
19 def curate_claims():
20     ...

```

We can use more than one Databricks Notebook to declare our DLT tables. Assuming we follow the **medallion architecture**, we can, for example, use different notebooks to define tables comprising the bronze, silver, and gold layers. The DLT framework can digest instructions defined across multiple notebooks to create a single workflow; all inter-table dependencies and relationships are processed and considered automatically. Figure 10 shows the complete workflow for our claims example. Starting with three source tables, DLT builds a comprehensive pipeline that delivers thirteen tables for business consumption.

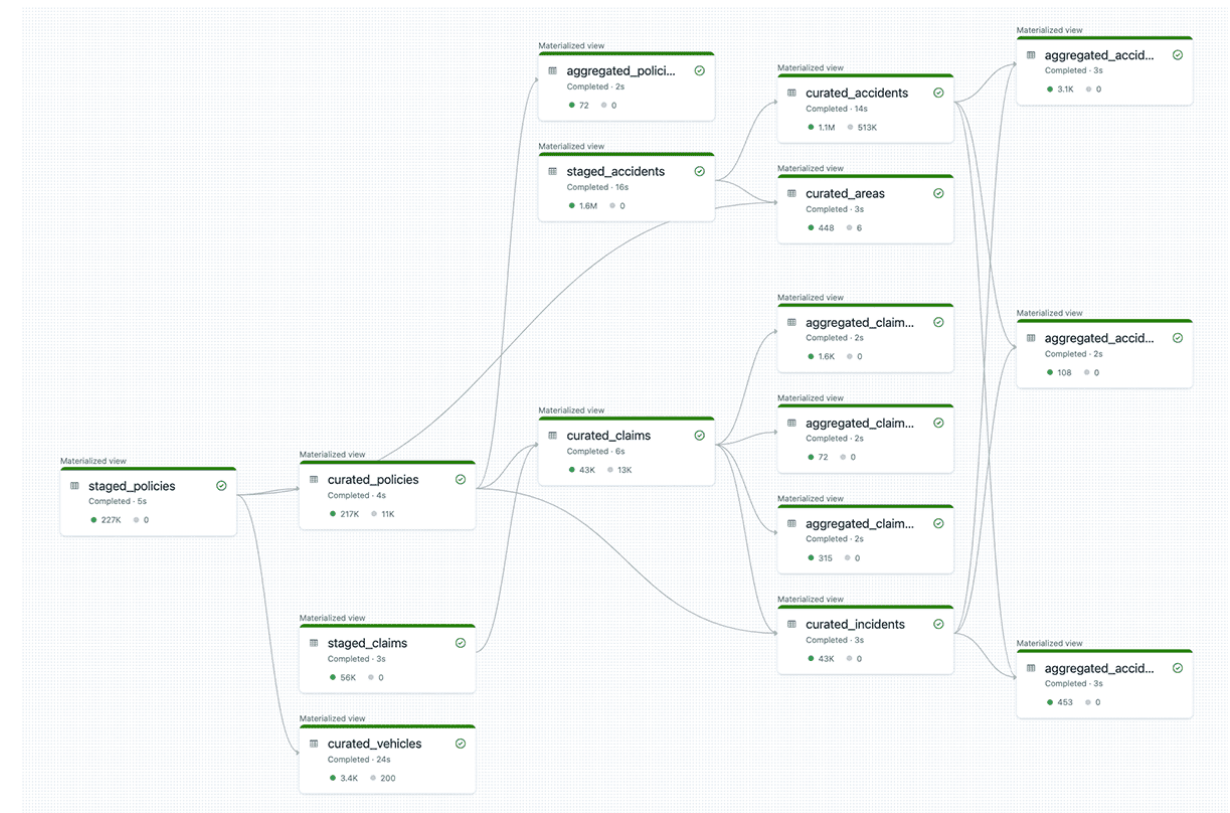


Figure 10: Overview of a complete Delta Live Tables (DLT) workflow

Results for each table can be inspected by selecting the desired entity. Figure 11 provides an example of the results of the curated claims table. DLT provides a high-level overview of the results from the data quality controls:

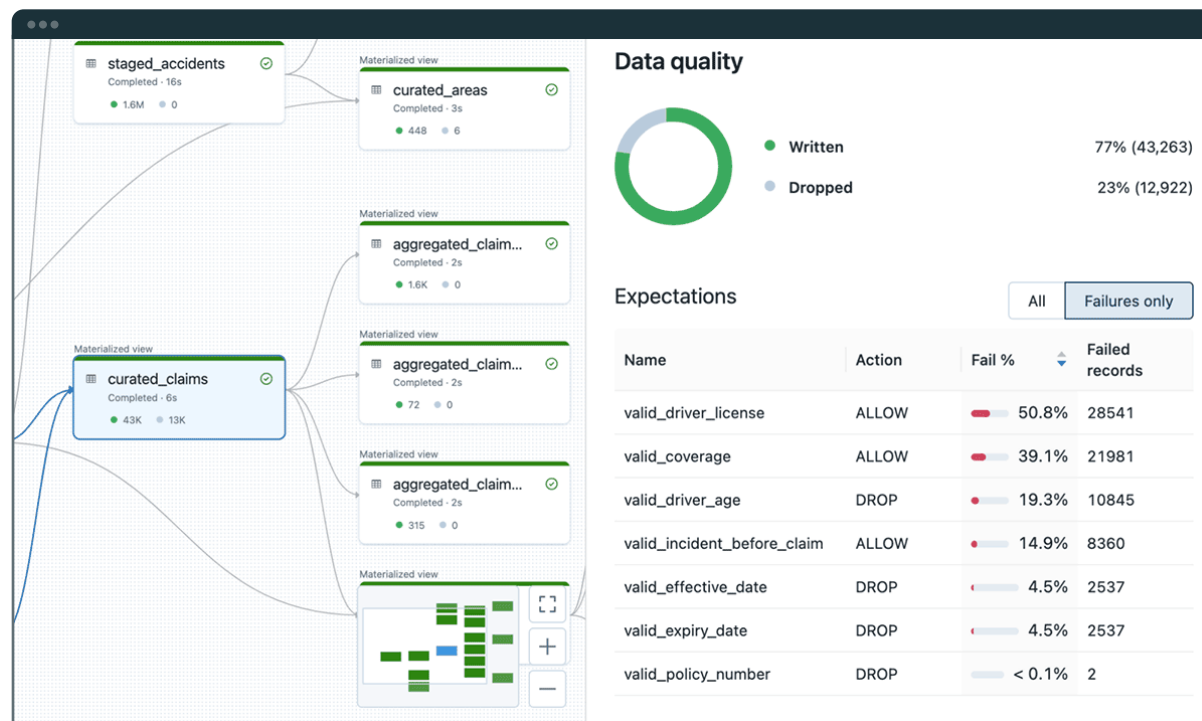


Figure 11: Example of detailed view for a Delta Live Tables (DLT) table entity with the associated data quality report

Results from the data quality expectations can be analyzed further by querying the **event log**. The event log contains detailed metrics about all expectations defined for the workflow pipeline. The query below provides an example for viewing key metrics from the last pipeline update, including the number of records that passed or failed expectations:

```

1  SELECT
2    row_expectations.dataset AS dataset,
3    row_expectations.name AS expectation,
4    SUM(row_expectations.passed_records) AS passing_records,
5    SUM(row_expectations.failed_records) AS failing_records
6  FROM
7    (
8      SELECT
9        explode(
10         from_json(
11           details :flow_progress :data_quality :expectations,
12           "array<struct<name: string, dataset: string, passed_records: int, failed_
13 records: int>>"
14         )
15       ) row_expectations
16     FROM
17       event_log_raw
18     WHERE
19       event_type = 'flow_progress'
20       AND origin.update_id = '${latest_update.id}'
21   )
22 GROUP BY
23   row_expectations.dataset,
24   row_expectations.name;

```

Again, we can view the complete history of changes made to each DLT table by looking at the Delta history logs (see Figure 12). It allows us to understand how tables evolve over time and investigate complete threads of updates if a pipeline fails.

Version	Timestamp	User Id	User Name	Operation	Operation Parameters	Job	Notebook	Cluster Id	Read Version	Isolation Level	Is Blind Append	Operation Metrics	User Metadata	Engine Info
12	2023-01-22T19:10:09	Null	Null	WRITE	> { ... } // 1 item	Null	Null	Null	11	WriteSerializable	false	> { ... } // 3 items	Null	Databricks- Runtime/dlt:11.0-delta-pipelines-1eca0d9-750b289-9ea72db-custom-local
11	2022-12-09T11:48:23	Null	Null	WRITE	> { ... } // 1 item	Null	Null	Null	10	WriteSerializable	false	> { ... } // 3 items	Null	Databricks- Runtime/dlt:11.0-delta-pipelines-ed5cc83-e81c5c7-17c692e-custom-local
10	2022-11-08T19:48:31	Null	Null	WRITE	> { ... } // 1 item	Null	Null	Null	9	WriteSerializable	false	> { ... } // 3 items	Null	Databricks- Runtime/dlt:11.0-delta-pipelines-de92f9e-8a33b70-a333f54-custom-local

Figure 12: View the history of changes made to a resulting Delta Live Tables (DLT) table entity

We can further use change data capture (CDC) to update tables based on changes in the source datasets. DLT CDC supports updating tables with slow-changing dimensions (SCD) types 1 and 2.

We have one of two options for our batch process to trigger the DLT pipeline. We can use the Databricks **Auto Loader** to incrementally process new data as it arrives in the source tables or create scheduled jobs that trigger at set times or intervals. In this example, we opted for the latter with a scheduled job that executes the DLT pipeline every five minutes.

OPERATIONALIZING THE OUTPUTS

The ability to incrementally process data efficiently is only half of the equation. Results from the DLT workflow must be operationalized and delivered to business users. In our example, we can consume outputs from the DLT pipeline through ad hoc analytics or prepacked insights made available through an interactive dashboard.

AD HOC ANALYTICS

Databricks SQL (or DB SQL) provides an efficient, cost-effective data warehouse on top of the Data Intelligence Platform. It allows us to run our SQL workloads directly against the source data with up to 12x better price/performance than its alternatives.

We can leverage DB SQL to perform specific ad hoc queries against our curated and aggregated tables. We might, for example, run a query against the curated policies table that calculates the total exposure. The DB SQL query editor provides a simple, easy-to-use interface to build and execute such queries (see example below).

```

1  SELECT
2      round(curr.total_exposure, 0) AS total_exposure,
3      round(prev.total_exposure, 0) AS previous_exposure
4  FROM
5      (
6          SELECT
7              sum(sum_insured) AS total_exposure
8          FROM
9              insurance_demo_lakehouse.curated_policies
10         WHERE
11             expiry_date > '{{ date.end }}'
12             AND (effective_date <= '{{ date.start }}'
13                 OR (effective_date BETWEEN '{{ date.start }}' AND '{{ date.end }}'))
14         ) curr
15     JOIN
16     (
17         SELECT
18             ...
19     ) prev

```

We can also use the DB SQL query editor to run queries against different versions of our Delta tables. For example, we can query a view of the aggregated claims records for a specific date and time (see example below). We can further use DB SQL to compare results from different versions to analyze only the changed records between those states.

```
1 SELECT
2 *
3 FROM
4 insurance_demo_lakehouse.aggregated_claims_weekly TIMESTAMP AS OF '2022-06-
5 05T17:00:00';
```

DB SQL offers the option to use a serverless compute engine, eliminating the need to configure, manage or scale cloud infrastructure while maintaining the lowest possible cost. It also integrates with alternative SQL workbenches (e.g., DataGrip), allowing analysts to use their favorite tools to explore the data and generate insights.

BUSINESS INSIGHTS

Finally, we can use DB SQL queries to create rich visualizations on top of our query results. These visualizations can then be packaged and served to end users through interactive dashboards (see Figure 13).

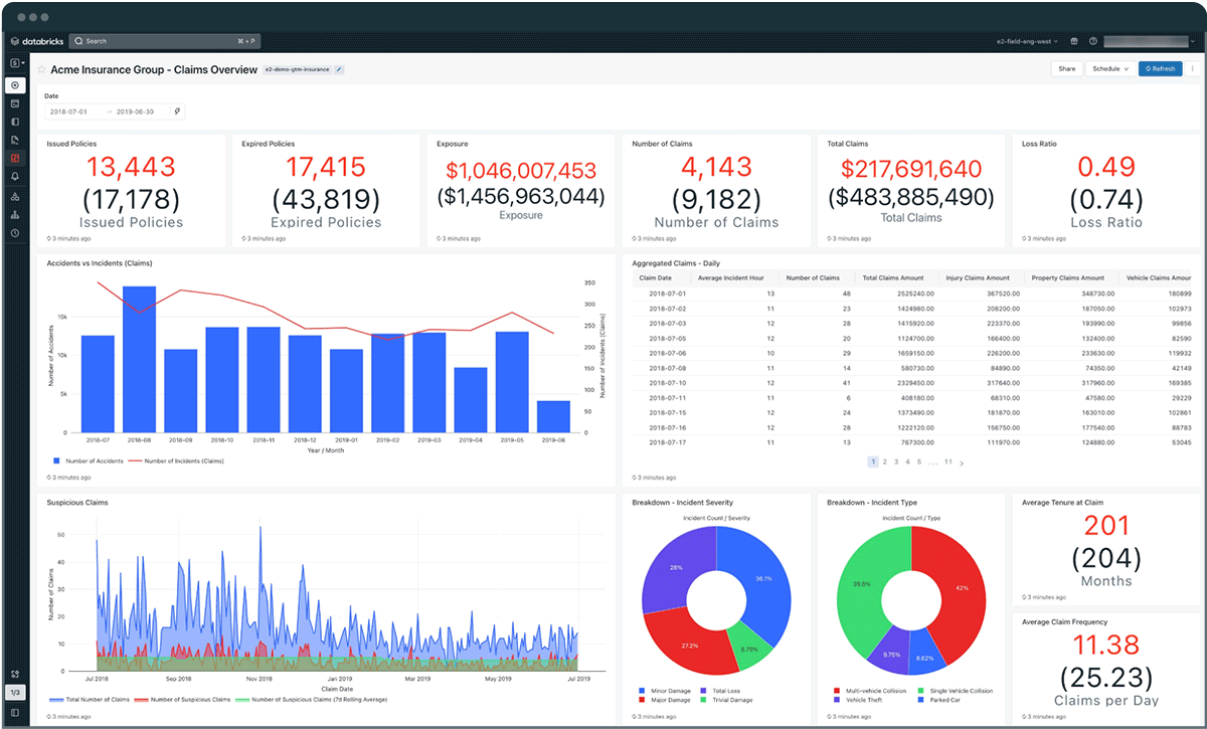


Figure 13: Example operational dashboard built on a set of resulting Delta Live Table (DLT) table entities

For our use case, we created a dashboard with a collection of key metrics, rolling calculations, high-level breakdowns, and aggregate views. The dashboard provides a complete summary of our claims process at a glance. We also added the option to specify specific date ranges. DB SQL supports a range of query parameters that can substitute values into a query at runtime. These query parameters can be defined at the dashboard level to ensure all related queries are updated accordingly.

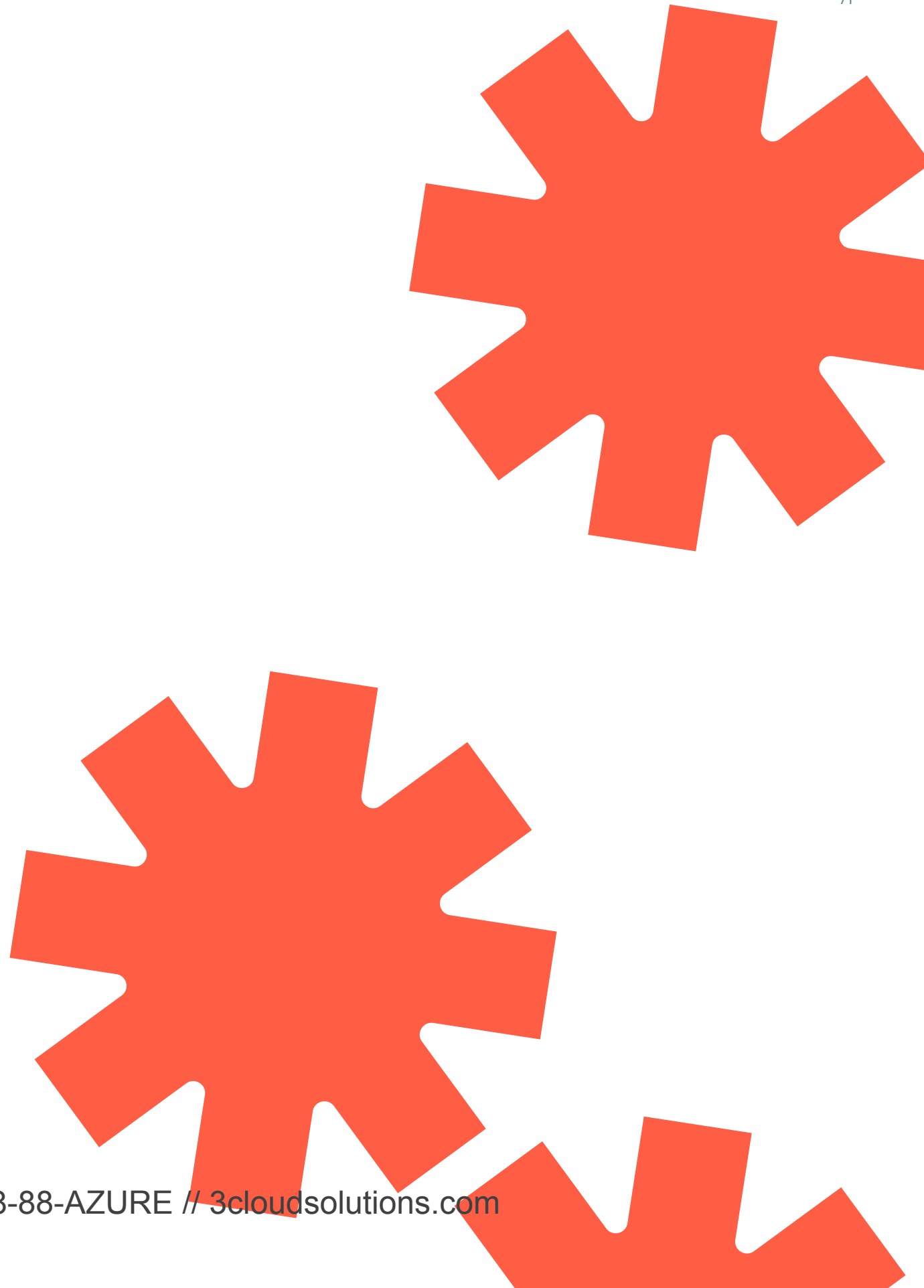
DB SQL integrates with numerous third-party analytical and BI tools like Power BI, Tableau and Looker. Like we did for Fivetran, we can use Partner Connect to link our external platform with DB SQL. This allows analysts to build and serve dashboards in the platforms that the business prefers without sacrificing the performance of DB SQL and the Databricks Lakehouse.

CONCLUSION

As we move into this fast-paced, volatile modern world of finance, batch processing remains a vital part of the modern data stack, able to hold its own against the features and benefits of streaming and real-time services.

We've seen how we can use the Databricks Data Intelligence Platform for Financial Services and its ecosystem of partners to architect a simple, scalable and extensible framework that supports complex batch-processing workloads with a practical example in insurance claims processing. With Delta Live Tables (DLT) and Databricks SQL (DB SQL), we can build a data platform with an architecture that scales infinitely, is easy to extend to address changing requirements and will withstand the test of time.

To learn more about the sample pipeline described, including the infrastructure setup and configuration used, please refer to [this](#) GitHub repository or watch [this](#) demo video.



How to Set Up Your First Federated Lakehouse

by Mike Dobing

Lakehouse Federation in Databricks is a groundbreaking new capability that allows you to query data across **external data sources** — including Snowflake, Synapse, many others and even Databricks itself — without having to move or copy the data. This is done by using Databricks **Unity Catalog**, which provides a unified metadata layer for all of your data.

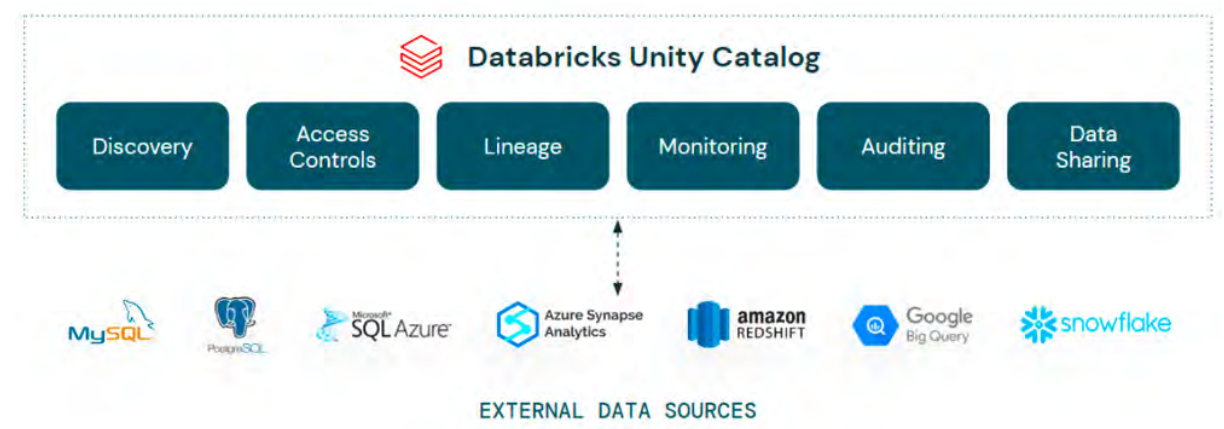
Lakehouse Federation is a game-changer for data teams, as it breaks down the silos that have traditionally kept data locked away in different systems. With Lakehouse Federation, you can finally access all of your data in one place, making it easier to get the insights you need to make better business decisions.

As always, though, not one solution is a silver bullet for your data integration and querying needs. See below for when Federation is a good fit, and for when you'd prefer to bring your data into your solution and process as part of your lakehouse platform pipelines.

A few of the benefits of using Lakehouse Federation in Databricks are:

- **Improved data access and discovery:** Lakehouse Federation makes it easy to find and access the data you need from your database estate. This is especially important for organizations with complex data landscapes.
- **Reduced data silos:** Lakehouse Federation can help to break down data silos by providing a unified view of all data across the organization.
- **Improved data governance:** Lakehouse Federation can help to improve data governance by providing a single place to manage permissions and access to data from within Databricks.
- **Reduced costs:** Lakehouse Federation can help to reduce costs by eliminating the need to move or copy data between different data sources.

If you are looking for a way to improve the way you access and manage your data across your analytics estate, then Lakehouse Federation in Databricks is a top choice.



REALITY CHECK

While Lakehouse Federation is a powerful tool, it is not a good fit for all use cases. There are some specific examples of use cases when Lakehouse Federation is not a good choice:

- **Real-time data processing:** Lakehouse Federation queries can be slower than queries on data that is stored locally in the lake. Therefore, Lakehouse Federation is not a good choice for applications that require real-time data processing.
- **Complex data transformations:** Where you need complex data transformations and processing, or need to ingest and transform vast amounts of data. For probably the large majority of use cases, you will need to apply some kind of ETL/ELT process against your data to make it fit for consumption by end users. In these scenarios, it is still best to apply a medallion style approach and bring the data in, process it, clean it, then model and serve it so it is performant and fit for consumption by end users.

Therefore, while Lakehouse Federation is a great option for certain use cases as highlighted above, it's not a silver bullet for all scenarios. Consider it an augmentation of your analytics capability that allows for additional use cases that need agility and direct source access for creating a holistic view of your data estate, all controlled through one governance layer.

SETTING UP YOUR FIRST FEDERATED LAKEHOUSE

With that in mind, let's get started on setting up your first federated lakehouse in Databricks using Lakehouse Federation.

For this example, we will be using a familiar sample database — AdventureWorks — running on an Azure SQL Database. We will be walking you through how to set up your connection to Azure SQL and how to add it as a foreign catalog inside Databricks.

PREREQUISITES

To set up Lakehouse Federation in Databricks, you will need the following prerequisites:

- [A Unity Catalog-enabled Databricks workspace with Databricks Runtime 13.1 or above and shared or sin...](#)
- [A Databricks Unity Catalog metastore](#)
- Network connectivity from your Databricks Runtime cluster or SQL warehouse to the target database systems, including any firewall connectivity requirements, such as [here](#)
- [The necessary permissions to create connections and foreign catalogs in Databricks Unity Catalog](#)
- The SQL Warehouse must be Pro or Serverless
- For this demo — an example database such as AdventureWorks to use as our data source, along with the necessary credentials. Please see this [example](#).

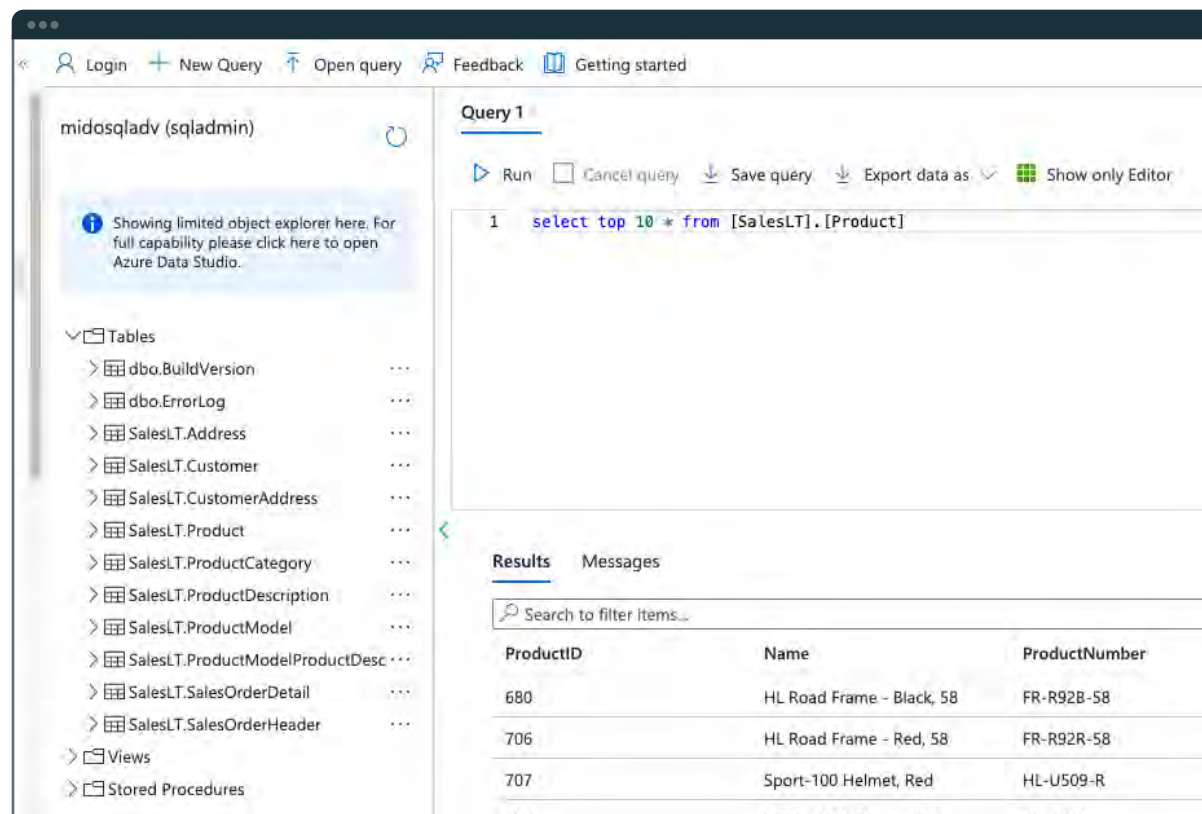
SETUP

Setting up federation is essentially a three-step process, as follows:

- Set up a connection
- Set up a foreign catalog
- Query your data sources

SETTING UP A CONNECTION

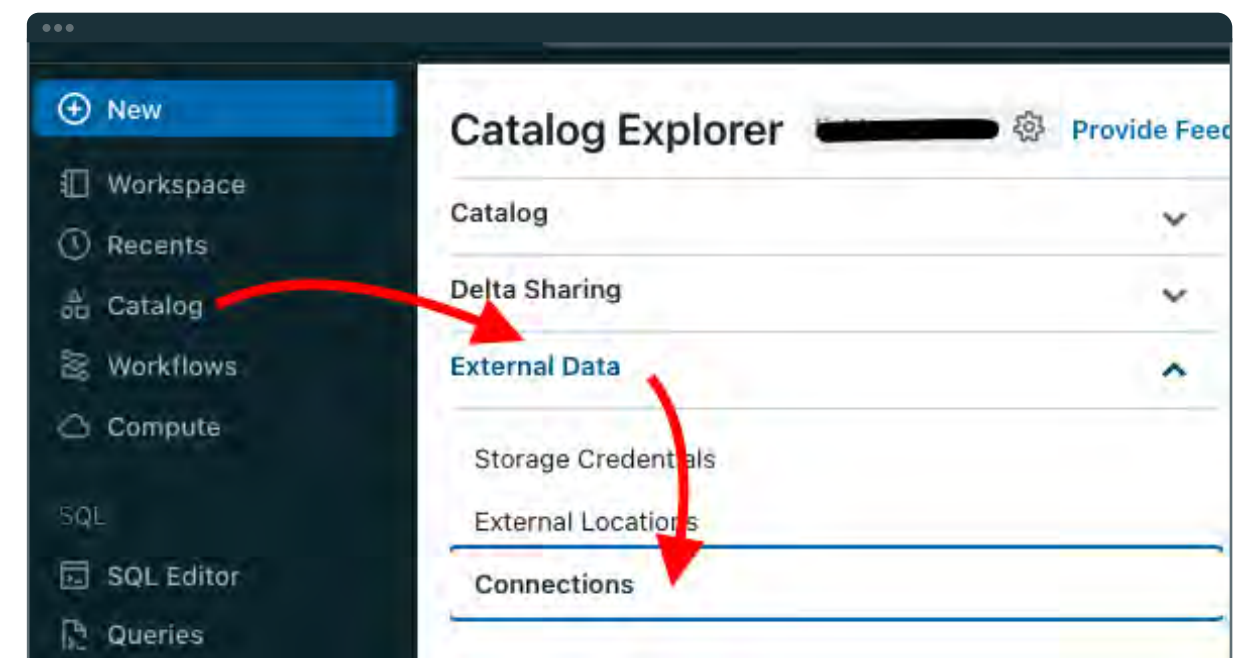
We are going to use Azure SQL Database as the test data source with the sample database AdventureWorksLT database already installed and ready to query:



We want to add this database as a foreign catalog in Databricks to be able to query it alongside other data sources. To connect to the database, we need a username, password and hostname, obtained from my Azure SQL Instance.

With these details ready, we can now go into Databricks and add the connection there as our first step.

First, expand the Catalog view, go to Connections and click "Create Connection":



Example query on the source database

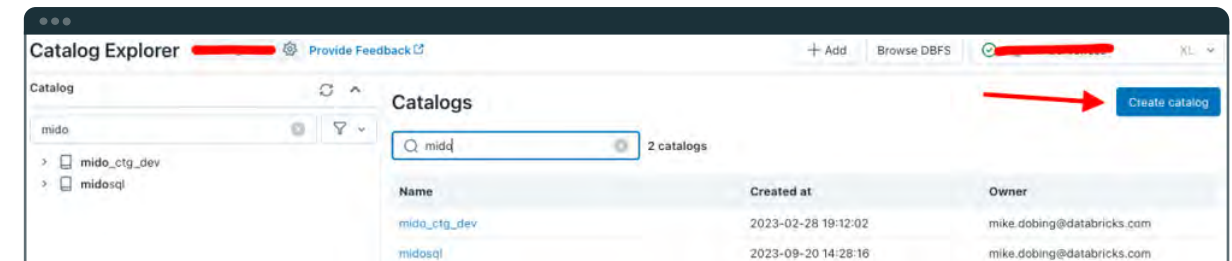
To add your new connection, give it a name, choose your connection type and then add the relevant login details for that data source:

The screenshot shows the 'Create a new connection' dialog box. It has the following fields and options:

- Connection name:** A text input field containing 'midosql'.
- Connection type:** A dropdown menu with 'SQL Server' selected.
- User:** A text input field with a cursor.
- Password:** A password input field.
- Host:** A text input field containing 'localhost.com'.
- Comment:** A text input field.
- Advanced options:** A link to expand more options.
- Buttons:** 'Test connection', 'Cancel', and 'Create'.

CREATE A FOREIGN CATALOG

Test your connection and verify all is well. From there, go back to the Catalog view and go to Create Catalog:

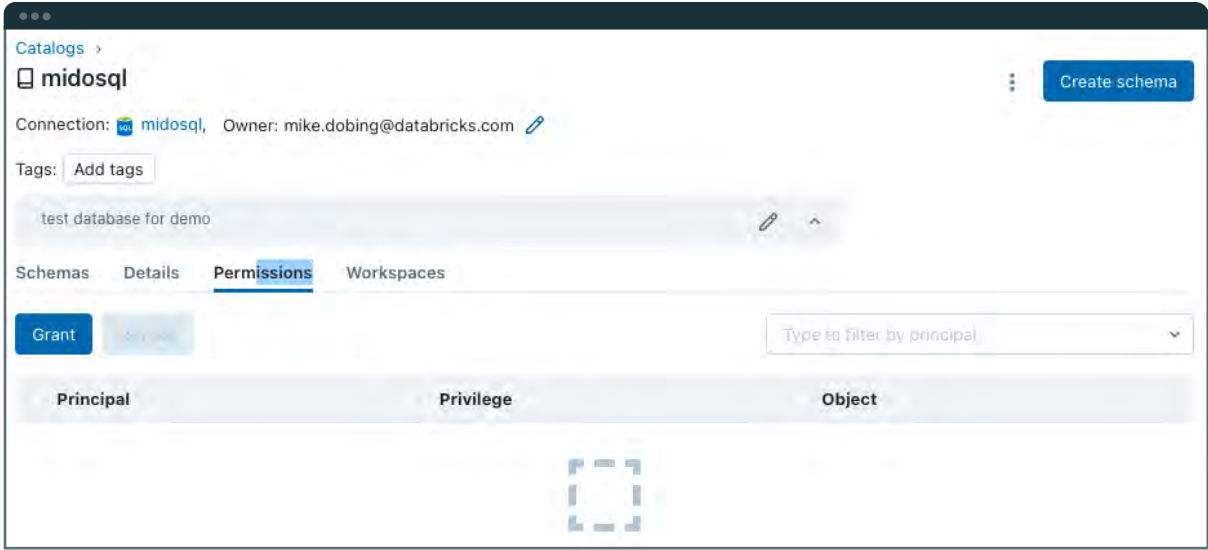


From there, populate the relevant details (choosing Type as "Foreign"), including choosing the connection you created in the first step, and specifying the database you want to add as an external catalog:

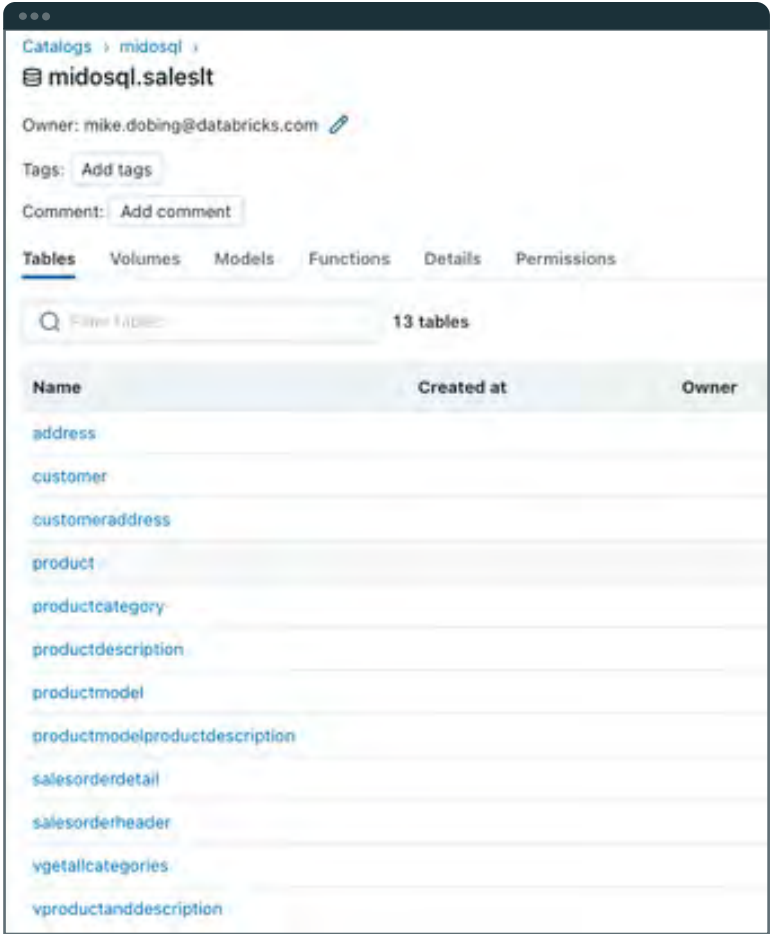
The screenshot shows the 'Create a new catalog' dialog box. It has the following fields and options:

- Catalog name:** A text input field containing 'midosql'.
- Type:** A dropdown menu with 'Foreign' selected.
- Connection:** A dropdown menu with 'midosql' selected.
- Database:** A text input field containing 'XXXX'.
- Comment (optional):** A text input field containing 'test database for demo'.
- Buttons:** 'Cancel' and 'Create'.

Once added, you can have the option of adding the relevant user permissions to the objects here, all governed by Unity Catalog (skipped this in this article as there are no other users using this database):

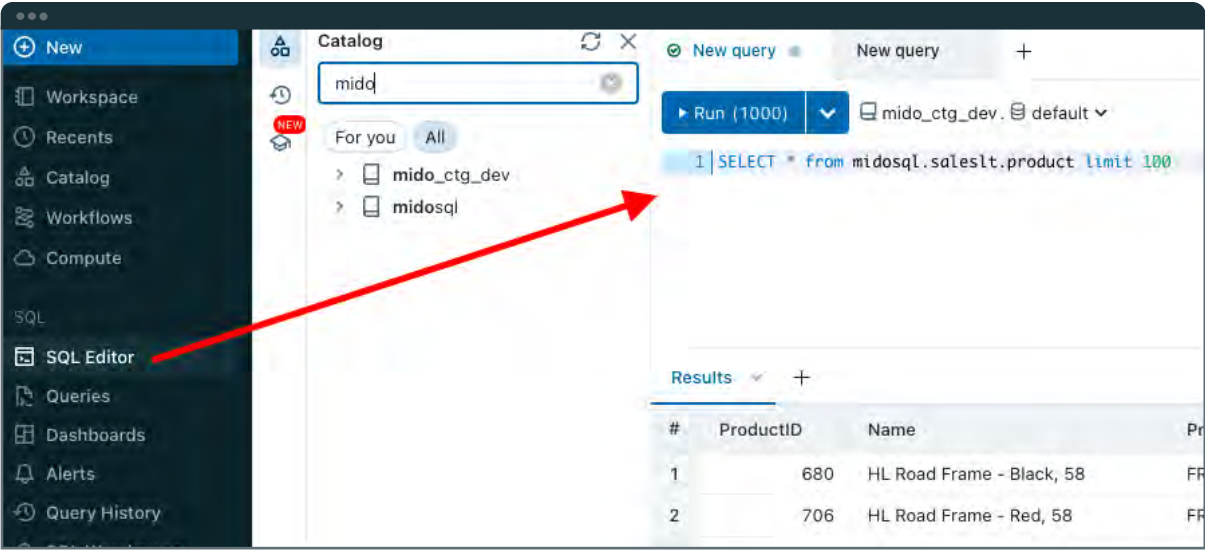


Our external catalog is now available for querying as you would any other catalog inside Databricks, bringing our broader data estate into our lakehouse:

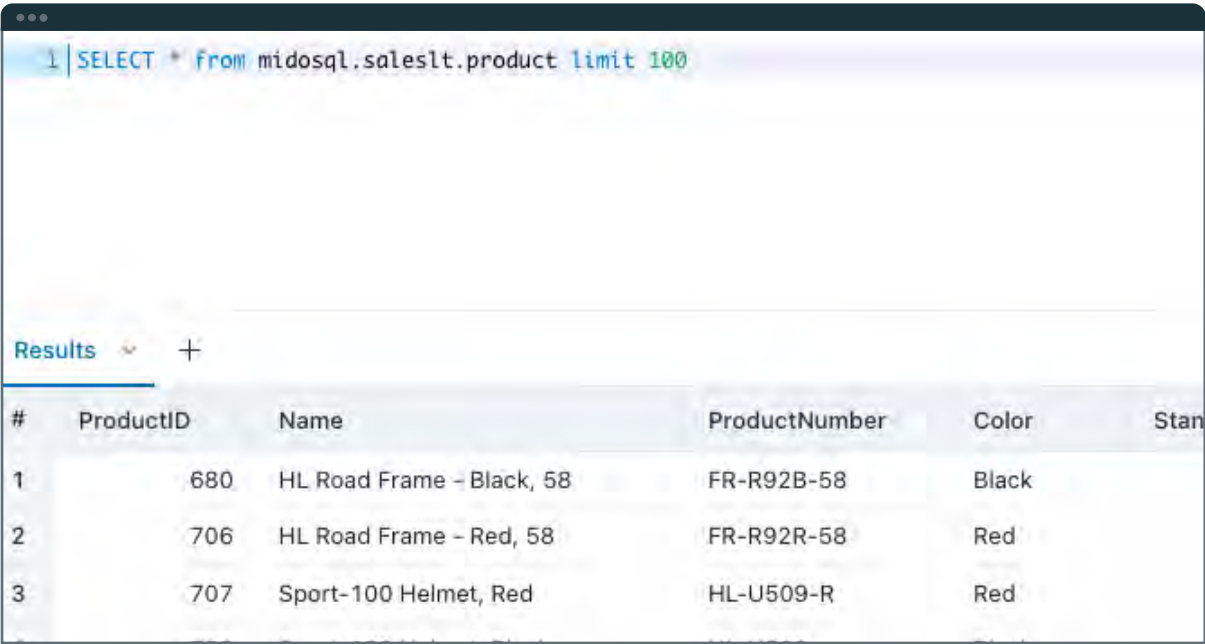


QUERYING THE FEDERATED DATA

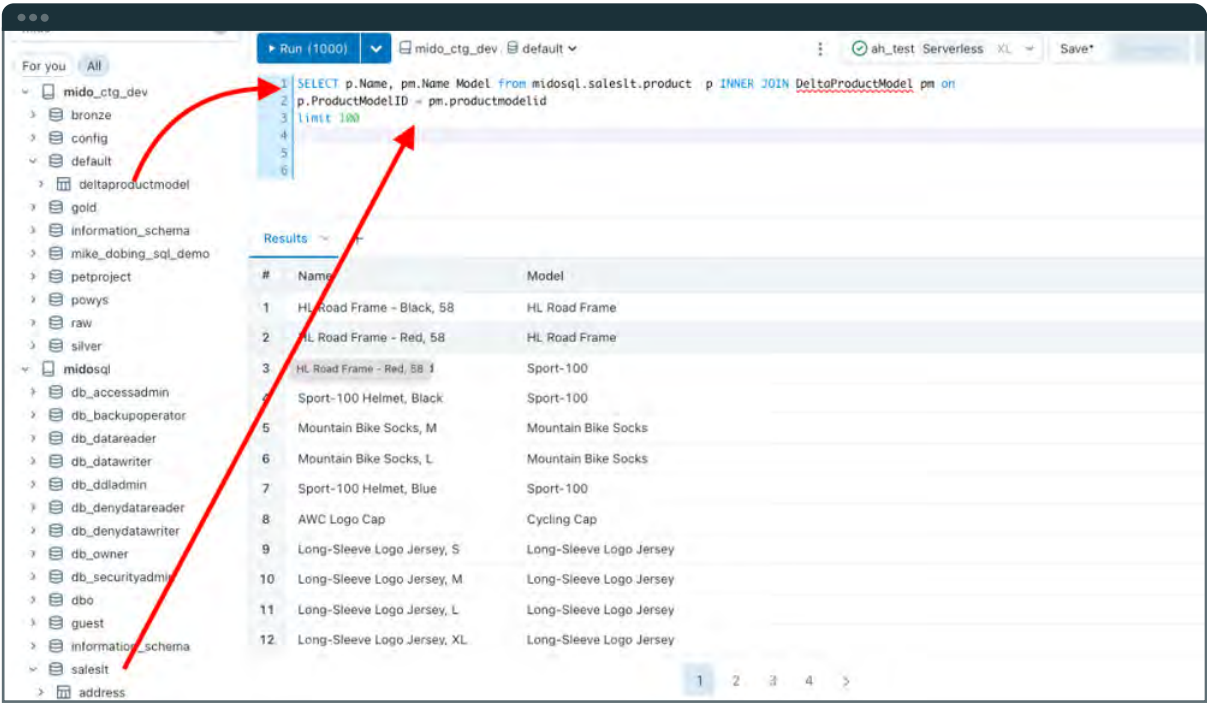
We can now access our federated Azure SQL Database as normal, straight from our Databricks SQL Warehouse:



And query it as we would any other object:



Or even join it to a local Delta table inside our Unity Catalog:



CONCLUSION

What we've shown here is just scratching the surface of what Lakehouse Federation can do with a simple connection and query. By leveraging this offering, combined with the governance and capabilities of Unity Catalog, you can extend the range of your lakehouse estate, ensuring consistent permissions and controls across all of your data sources and thus enabling a plethora of new use cases and opportunities.

Orchestrating Data Analytics With Databricks Workflows

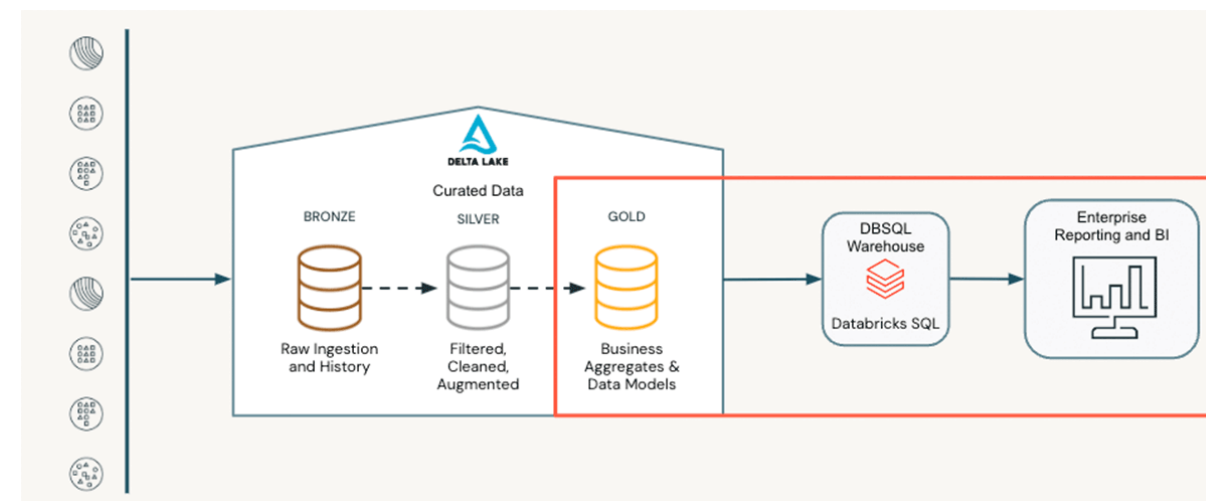
by Matthew Kuehn

For data-driven enterprises, data analysts play a crucial role in extracting insights from data and presenting it in a meaningful way. However, many analysts might not have the familiarity with data orchestration required to automate their workloads for production. While a handful of ad hoc queries can quickly turn around the right data for a last-minute report, data teams must ensure that various processing, transformation and validation tasks are executed reliably and in the right sequence. Without the proper orchestration in place, data teams lose the ability to monitor pipelines, troubleshoot failures and manage dependencies. As a result, sets of ad hoc queries that initially brought quick-hitting value to the business end up becoming long-term headaches for the analysts who built them.

Pipeline automation and orchestration becomes particularly crucial as the scale of data grows and the complexity of pipelines increases. Traditionally, these responsibilities have fallen on data engineers, but as data analysts begin to develop more assets in the lakehouse, orchestration and automation becomes a key piece to the puzzle.

For data analysts, the process of querying and visualizing data should be seamless, and that's where the power of modern tools like **Databricks Workflows** comes into play. In this chapter, we'll explore how data analysts can leverage Databricks Workflows to automate their data processes, enabling them to focus on what they do best — deriving value from data.

THE DATA ANALYST'S WORLD

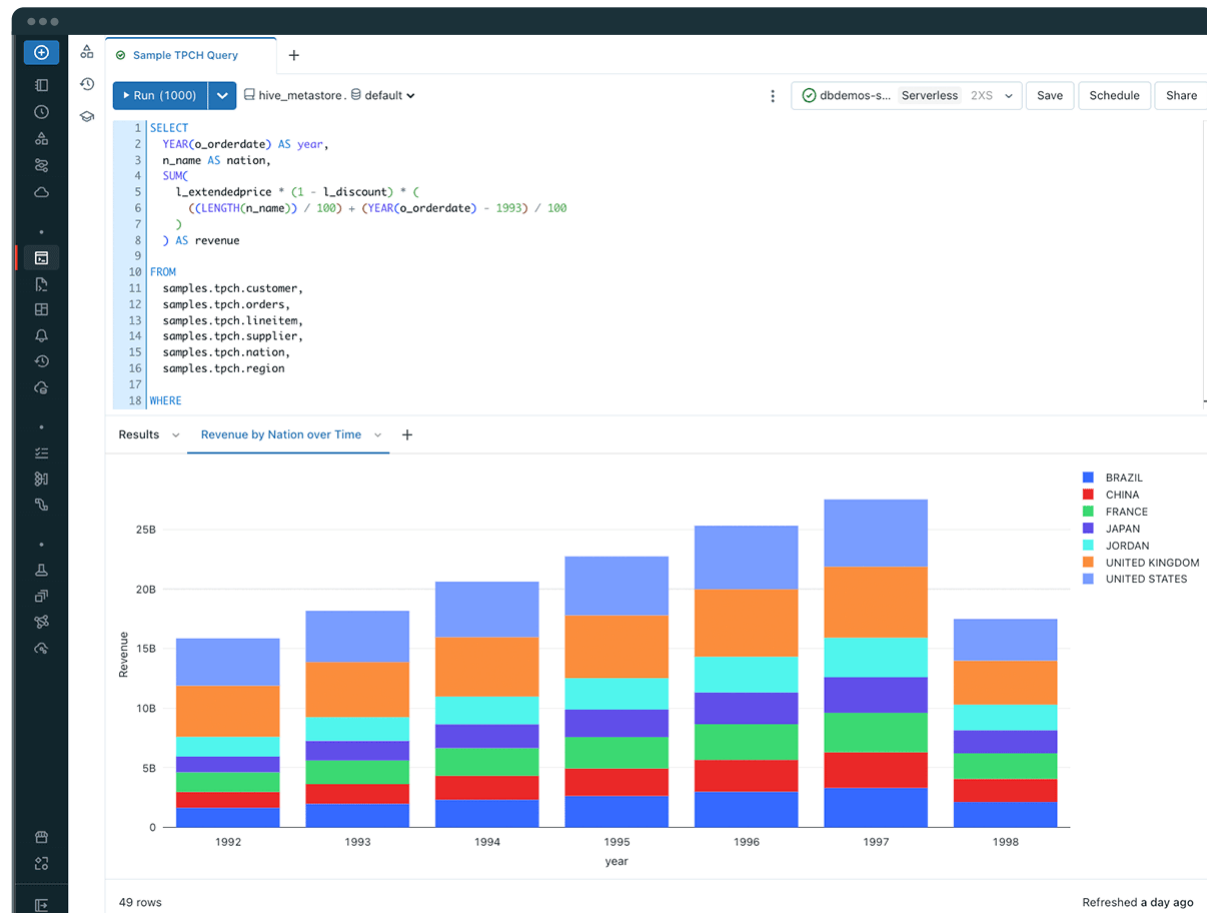


Data analysts play a vital role in the final stages of the data lifecycle. Positioned at the "last mile," they rely on refined data from upstream pipelines. This could be a table prepared by a data engineer or the output predictions of machine learning models built by data scientists. This refined data, often referred to as the Silver layer in a medallion architecture, serves as the foundation for their work. Data analysts are responsible for aggregating, enriching and shaping this data to answer specific questions for their business, such as:

- "How many orders were placed for each SKU last week?"
- "What was monthly revenue for each store last fiscal year?"
- "Who are our 10 most active users?"

These aggregations and enrichments build out the Gold layer of the medallion architecture. This Gold layer enables easy consumption and reporting for downstream users, typically in a visualization layer. This can take the form of dashboards within Databricks or be seamlessly generated using external tools like Tableau or Power BI via **Partner Connect**. Regardless of the tech stack, data analysts transform raw data into valuable insights, enabling informed decision-making through structured analysis and visualization techniques.

THE DATA ANALYST'S TOOLKIT ON DATABRICKS



In Databricks, data analysts have a robust toolkit at their fingertips to transform data effectively on the lakehouse. Centered around the **Databricks SQL Editor**, analysts have a familiar environment for composing ANSI SQL queries, accessing data and exploring table schemas. These queries serve as building blocks for various SQL assets, including visualizations that offer in-line data insights. **Dashboards** consolidate multiple visualizations, creating a user-friendly interface for comprehensive reporting and data exploration for end users.

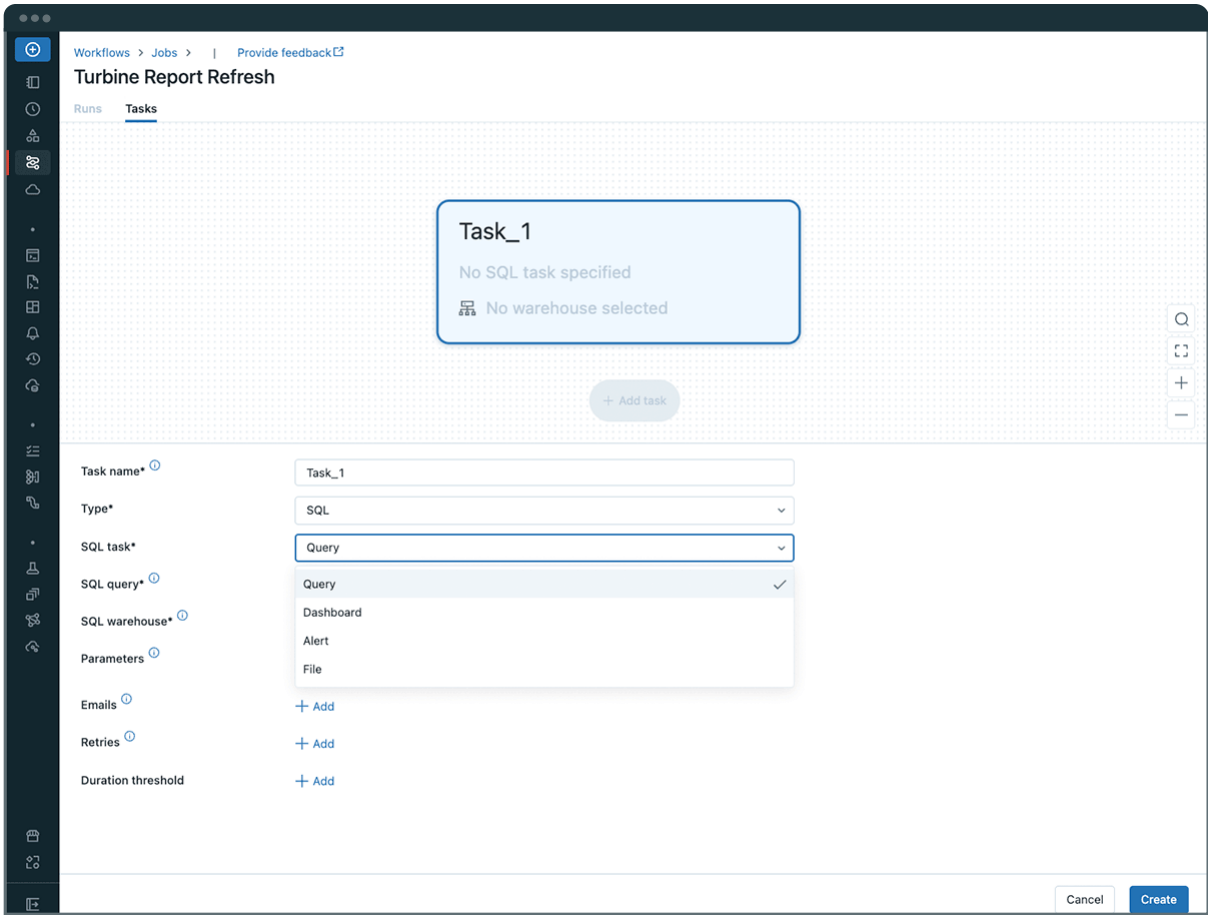
Additionally, **alerts** keep analysts informed about critical dataset changes in real time. Serverless SQL Warehouses are underpinning all these features, which can scale to handle diverse data volumes and query demands. By default, this compute uses **Photon**, the high-performance Databricks-native vectorized query engine, and is optimized for high-concurrency SQL workloads. Finally, **Unity Catalog** allows users to easily govern structured and unstructured data, machine learning models, notebooks, dashboards and files in the lakehouse. This cohesive toolkit empowers data analysts to transform raw data into enriched insights seamlessly within the Databricks environment.

ORCHESTRATING THE DATA ANALYST'S TOOLKIT WITH WORKFLOWS

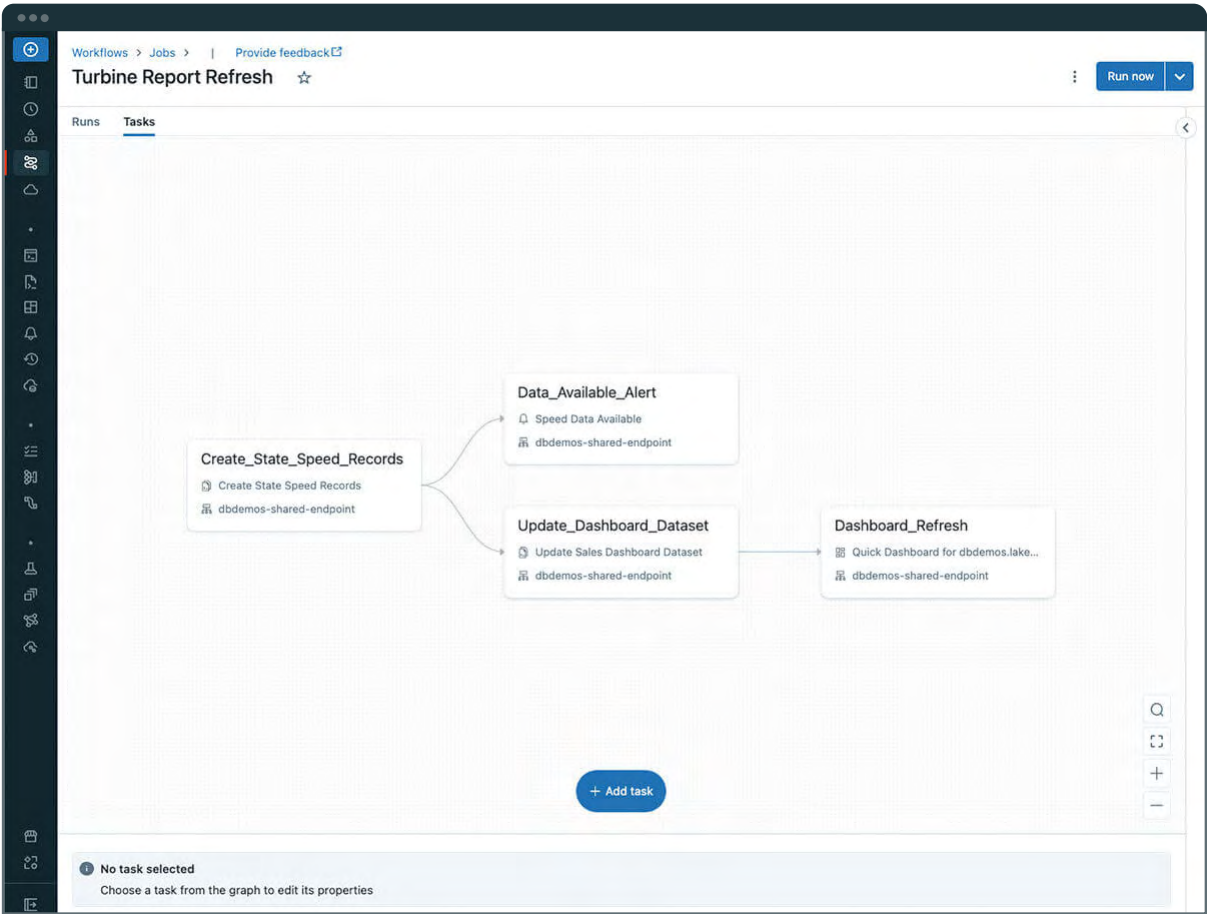
For those new to Databricks, Workflows orchestrates data processing, machine learning and analytics pipelines in the Data Intelligence Platform. Workflows is a fully managed orchestration service integrated with the Databricks Platform, with high reliability and advanced observability capabilities. This allows all users, regardless of persona or background, to easily orchestrate their workloads in production environments.

Authoring Your SQL Tasks

Building your first workflow as a data analyst is extremely simple. Workflows now seamlessly integrates the core tools used by data analysts — queries, alerts and dashboards — within its framework, enhancing its capabilities through the SQL task type. This allows data analysts to build and work with the tools they are already familiar with and then easily bring them into a Workflow as a Task via the UI.



As data analysts begin to chain more SQL tasks together, they will begin to easily define dependencies between and gain the ability to schedule and automate SQL-based tasks within Databricks Workflows. In the below example workflow, we see this in action:



Imagine that we have received upstream data from our data engineering team that allows us to begin our dashboard refresh process. We can define SQL-centric tasks like the ones below to automate our pipeline:

- **Create_State_Speed_Records:** First, we define our refreshed data in our Gold layer with the Query task. This inserts data into a Gold table and then optimizes it for better performance.
- **Data_Available_Alert:** Once this data is inserted, imagine we want to notify other data analysts who consume this table that new records have been added. We can do this by creating an alert which will trigger when we have new records added. This will send an alert to our stakeholder group. You can imagine using an alert in a similar fashion for data quality checks to warn users of stale data, null records or other similar situations. *For more information on creating your first alert, check out [this link](#).*
- **Update_Dashboard_Dataset:** It's worth mentioning that tasks can be defined in parallel if needed. In our example, while our alert is triggering we can also begin refreshing our tailored dataset view that feeds our dashboard in a parallel query.
- **Dashboard_Refresh:** Finally, we create a dashboard task type. Once our dataset is ready to go, this will update all previously defined visualizations with the newest data and notify all subscribers upon successful completion. Users can even pass specific parameters to the dashboard while defining the task, which can help generate a default view of the dashboard depending on the end user's needs.

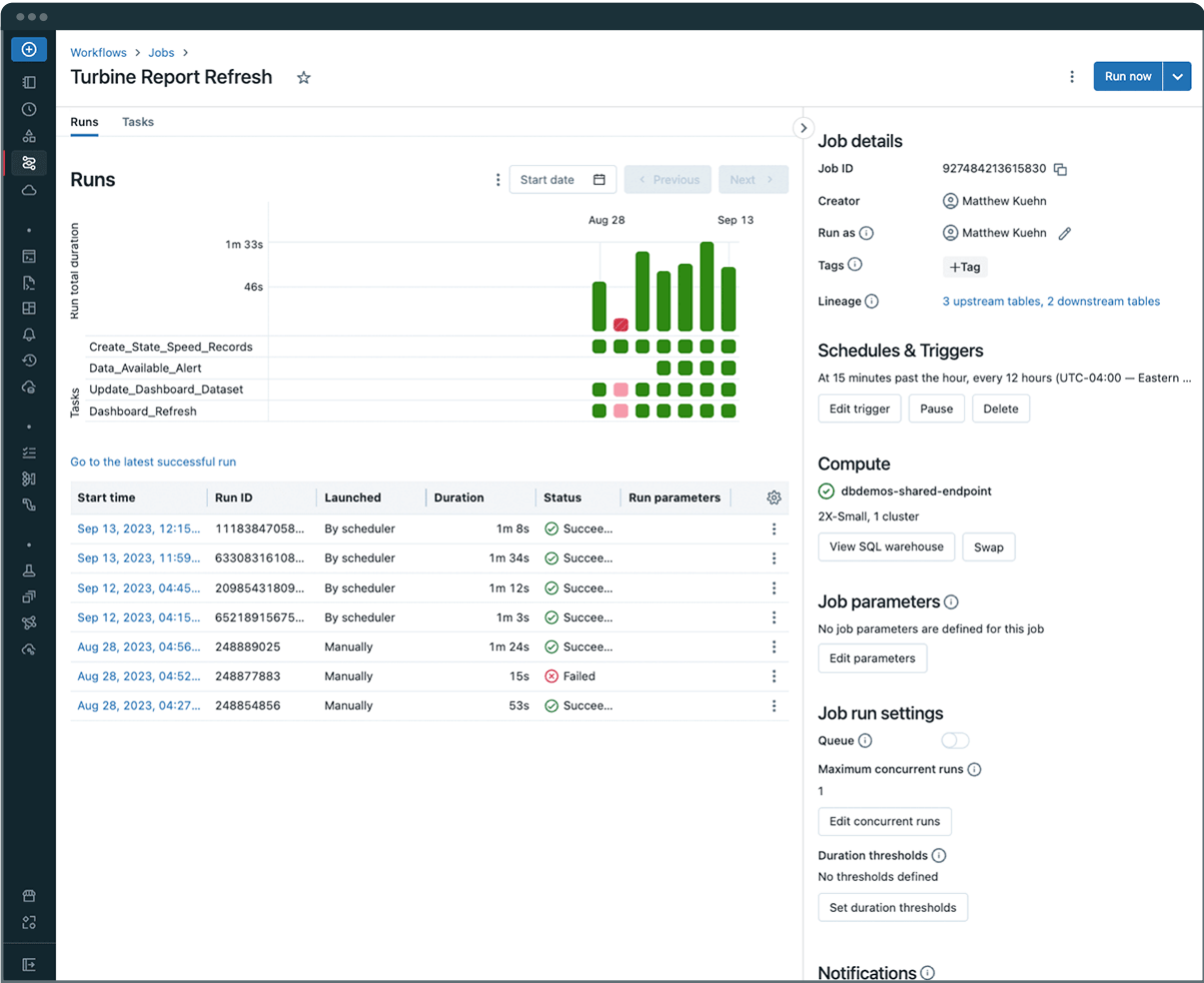
It is worth noting that this example workflow utilizes queries directly written in the Databricks SQL Editor. A similar pattern can be achieved with SQL code coming from a repository using the File task type. With this task type, users can execute .sql files stored in a Git repository as part of an automated workflow. Each time the pipeline is executed, the latest version from a specific branch will be retrieved and executed.

Although this example is basic, you can begin to see the possibilities of how a data analyst can define dependencies across SQL task types to build a comprehensive analytics pipeline.



MONITORING YOUR PRODUCTION PIPELINES

While authoring is comprehensive within Databricks Workflows, it is only one part of the picture. Equally important is the ability to easily monitor and debug your pipelines once they are built and in production.



Databricks Workflows allows users to monitor individual job runs, offering insights into task outcomes and overall execution times. This visibility helps analysts understand query performance, identify bottlenecks and address issues efficiently. By promptly recognizing tasks that require attention, analysts can ensure seamless data processing and quicker issue resolution.

When it comes to executing a pipeline at the right time, Databricks Workflows allows users to schedule jobs for execution at specific intervals or trigger them when certain files arrive. In the above image, we were first manually triggering this pipeline to test and debug our tasks. Once we got this to a steady state, we began triggering this every 12 hours to accommodate for data refresh needs across time zones. This flexibility accommodates varying data scenarios, ensuring timely pipeline execution. Whether it's routine processing or responding to new data batches, analysts can tailor job execution to match operational requirements.

Late-arriving data can bring a flurry of questions to a data analyst from end users. Workflows enables analysts and consumers alike to stay informed on data freshness by setting up notifications for job outcomes such as successful execution, failure or even a long-running job. These notifications ensure timely awareness of changes in data processing. By proactively evaluating a pipeline's status, analysts can take proactive measures based on real-time information.

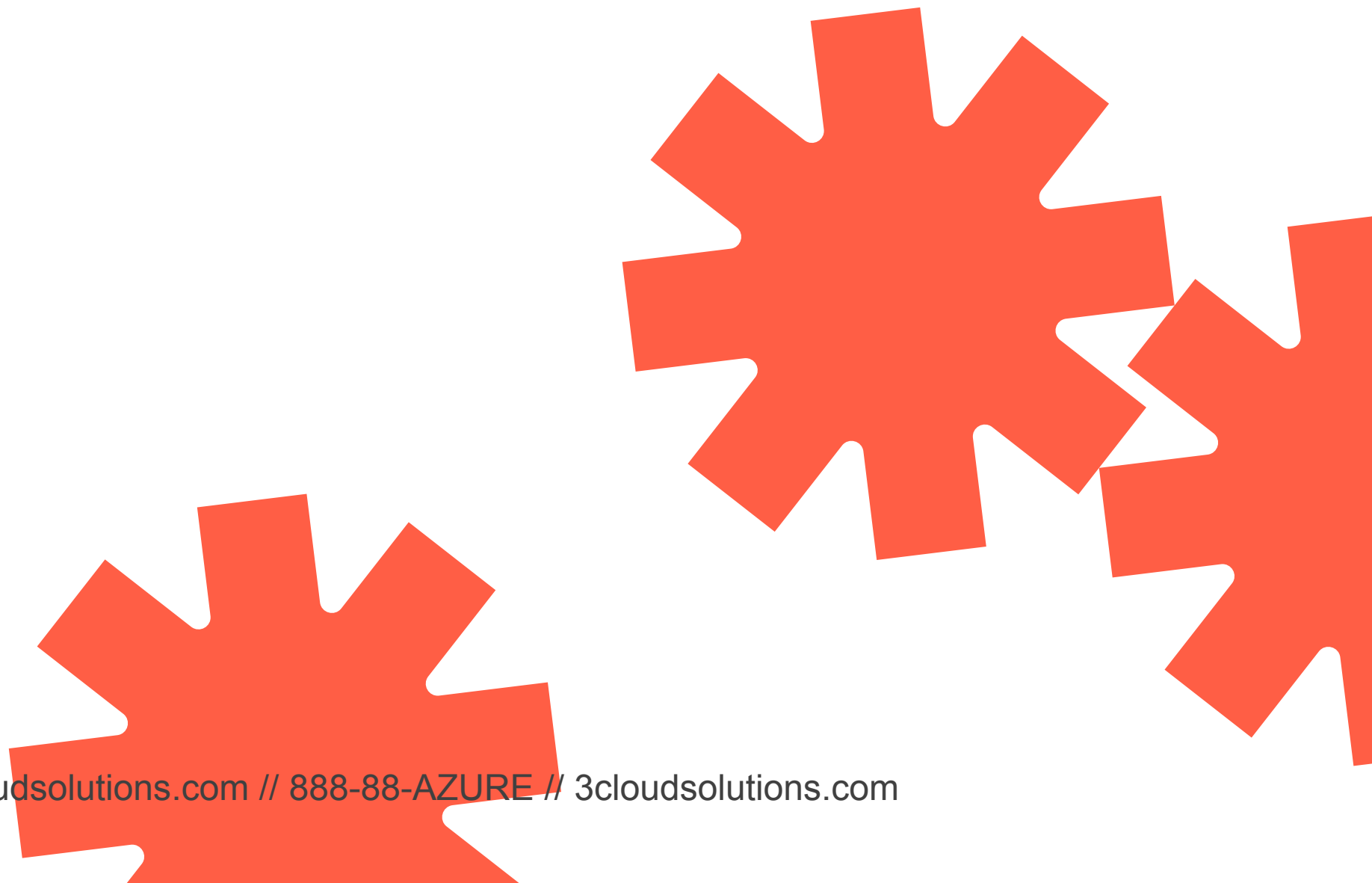
As with all pipelines, failures will inevitably happen. Workflows helps manage this by allowing analysts to configure job tasks for automatic retries. By automating retries, analysts can focus on generating insights rather than troubleshooting intermittent technical issues.

CONCLUSION

In the evolving landscape of data analysis tools, Databricks Workflows bridges the gap between data analysts and the complexities of data orchestration. By automating tasks, ensuring data quality and providing a user-friendly interface, Databricks Workflows empowers analysts to focus on what they excel at — extracting meaningful insights from data. As the concept of the lakehouse continues to unfold, Workflows stands as a pivotal component, promising a unified and efficient data ecosystem for all personas.

GET STARTED

- Learn more about [Databricks Workflows](#)
- Take a [product tour of Databricks Workflows](#)
- Create your first workflow with this [quickstart guide](#)



Schema Management and Drift Scenarios via Databricks Auto Loader

by Garrett Peternel

Data lakes notoriously have had challenges with managing incremental data processing at scale without integrating open table storage format frameworks (e.g., Delta Lake, Apache Iceberg, Apache Hudi). In addition, schema management is difficult with schema-less data and schema-on-read methods. With the power of the Databricks Platform, Delta Lake and Apache Spark provide the essential technologies integrated with Databricks Auto Loader (AL) to consistently and reliably stream and process raw data formats incrementally while maintaining stellar performance and data governance.

AUTO LOADER FEATURES

AL is a boost over Spark Structured Streaming, supporting several additional benefits and solutions including:

- Databricks Runtime only Structured Streaming cloudFiles source
- Schema drift, dynamic inference and evolution support
- Ingests data via JSON, CSV, PARQUET, AVRO, ORC, TEXT and BINARYFILE input file formats
- Integration with cloud file notification services (e.g., Amazon SQS/SNS)
- Optimizes directory list mode scanning performance to discover new files in cloud storage (e.g., AWS, Azure, GCP, DBFS)

For further information please visit the official [Databricks Auto Loader](#) documentation.

SCHEMA CHANGE SCENARIOS

In this chapter I will showcase a few examples of how AL handles schema management and drift scenarios using a public IoT sample dataset with schema modifications to showcase solutions. Schema 1 will contain an IoT sample dataset schema with all expected columns and expected data types. Schema 2 will contain unexpected changes to the IoT sample dataset schema with new columns and changed data types. The following variables and paths will be used for this demonstration along with [Databricks Widgets](#) to set your username folder.

```
1 %scala
2 dbutils.widgets.text("dbfs_user_dir", "your_user_name") // widget for account email
3
4 val userName = dbutils.widgets.get("dbfs_user_dir")
5 val rawBasePath = s"dbfs:/user/$userName/raw/"
6 val repoBasePath = s"dbfs:/user/$userName/repo/"
7
8 val jsonSchema1Path = rawBasePath + "iot-schema-1.json"
9 val jsonSchema2Path = rawBasePath + "iot-schema-2.json"
10 val repoSchemaPath = repoBasePath + "iot-ddl.json"
11
12 dbutils.fs.rm(repoSchemaPath, true) // remove schema repo for demos
```

SCHEMA 1

```
1 %scala
2 spark.read.json(jsonSchema1Path).printSchema
```